

Algoritmos y Estructuras de Datos 1

2025-01-01

Contents

1	Programación Orientada a Objetos	1
1.1	Conceptos esenciales	1
1.1.1	Tipos de Datos	1
1.1.2	Tipo de dato simple y compuesto	2
1.1.3	Tipo referencia y tipo valor	2
1.1.4	Tipos de Datos Abstractos	3
1.2	Conceptos esenciales de programación orientada a objetos (POO)	4
1.2.1	Definición de Objeto	4
1.2.2	Definición de Clase	4
1.3	Principios de programación orientada a objetos	5
1.3.1	Abstracción	5
1.3.2	Encapsulamiento	6
1.3.3	Herencia	8
1.3.4	Polimorfismo	9
2	Estructuras de Datos Lineales	13
2.1	Arreglos y Matrices	13
2.1.1	Arreglos	13
2.1.2	Matrices	15
2.2	Listas	18
2.2.1	Tipo de Dato Abstracto Lista	18
2.2.2	Implementación mediante arreglos	18
2.2.3	Implementación mediante listas simples enlazadas	20
2.2.4	Implementación mediante lista doblemente enlazada	23
2.2.5	Implementación mediante lista enlazada circular	25
2.2.6	Tabla comparativa de implementaciones	28
2.3	Pilas	28
2.3.1	Tipo de dato abstracto Pila	28
2.3.2	Implementación de pilas con arreglos	29
2.3.3	Implementación de pilas con listas enlazadas	30
2.4	Colas	31
2.4.1	Tipo de dato abstracto Cola	31
2.4.2	Implementación de colas con arreglos	32

2.4.3	Implementación de colas con listas enlazadas	34
2.5	Colas de prioridad	35
2.5.1	Tipo de dato abstracto Cola de prioridad	35
2.5.2	Tipos de colas de prioridad	36
2.5.3	Implementación de colas de prioridad con listas enlazadas	36
2.6	Generics	37
2.7	Referencias adicionales	39
3	Estructuras de datos jerárquicas	41
3.1	Terminología básica	42
3.2	Tipo de dato abstracto Árbol	42
3.3	Árboles Binarios de Búsqueda (BST)	43
3.3.1	Implementación	43
3.3.2	Búsqueda de elementos	43
3.3.3	Inserción de elementos	44
3.3.4	Eliminación de elementos	45
3.3.5	Recorridos	46
3.4	Árboles Heap (montículo)	48
3.4.1	Tipo de dato abstracto	49
3.4.2	Implementación de árboles heap	49
3.5	Árboles AVL	52
3.5.1	Características	52
3.5.2	Ventajas	53
3.5.3	Desventajas	53
3.5.4	Inserción	53
3.5.5	Eliminación	59
3.5.6	Búsquedas y recorridos	60
3.6	Árboles de expresión	61
3.6.1	Conversión de arboles de expresión a notación infija	61
3.6.2	Conversión de expresión a árbol de expresión	61
3.6.3	Convertir expresión infija a postfija	62
3.7	Árboles B	62
3.7.1	Características	62
3.7.2	Estructura básica en Java	63
3.7.3	Búsqueda	63
3.7.4	Inserción	64
3.7.5	Eliminación	65
3.8	Tries	69
3.8.1	El TDA Trie	69
3.8.2	Estructura básica	69
3.8.3	Inserción	74
3.8.4	Búsqueda	75
4	Algoritmos de búsqueda y ordenamiento	77
4.1	Algoritmos de búsqueda	77
4.1.1	Búsqueda lineal	77

4.1.2	Búsqueda binaria	78
4.1.3	Búsqueda Hash	79
4.2	Algoritmos de ordenamiento	81
4.2.1	Ordenamiento por selección	81
4.2.2	Bubble sort	82
4.2.3	Ordenamiento por inserción (Insertion sort)	82
4.2.4	Quick sort	82
4.2.5	Shellsort	82
4.2.6	Radix sort	84
5	Grafos	85
5.1	Conceptos importantes	86
5.2	Representación	86
5.2.1	Matriz de adyacencia	88
5.2.2	Lista de adyacencia	89
5.3	Recorridos de un grafo	89
5.3.1	Breadth-First	90
5.3.2	Depth-First	90
5.4	Camino más corto: Dijkstra	91
5.4.1	Estructura general	93
5.5	Camino más corto: Floyd	94
5.6	Warshall	96
5.7	Minimal Spanning tree	96
5.7.1	Prim	98
5.7.2	Kruskal	98

Chapter 1

Programación Orientada a Objetos

En este capítulo, se introduce la programación orientada a objetos junto a conceptos esenciales para comprender su funcionamiento e implementación en lenguajes de programación comunes.

1.1 Conceptos esenciales

Antes de entrar en detalle en POO, es importante entender algunos conceptos básicos de programación y tipos de datos. Estos conceptos son fundamentales para comprender cómo funciona la programación orientada a objetos y cómo se pueden utilizar en la práctica.

1.1.1 Tipos de Datos

Un *tipo de dato* es una clasificación que especifica qué tipo de valores puede tomar una variable, así como las operaciones que se pueden realizar sobre estos valores. Los tipos de datos se utilizan en la declaración de variables y en la definición de funciones, entre otros usos.

Por ejemplo, en `Python`, los tipos de datos más comunes son:

- **Enteros (`int`):** Números enteros como 1, 20, -5.
- **Flotantes (`float`):** Números con decimales como 3.14
- **Cadenas (`str`):** Secuencias de caracteres como “Hola, mundo!”
- **Booleanos (`bool`):** Valores de verdad como `True` o `False`

Con respecto a las operaciones que se pueden realizar con estos tipos de datos, por ejemplo, se pueden realizar operaciones aritméticas con enteros y flotantes, concatenar cadenas, y realizar operaciones lógicas con booleanos.

Los IDEs suelen proporcionar herramientas para trabajar con tipos de datos, como autocompletado y resaltado de sintaxis, lo que facilita la escritura y comprensión del código tal y como se muestra en la siguiente imagen:

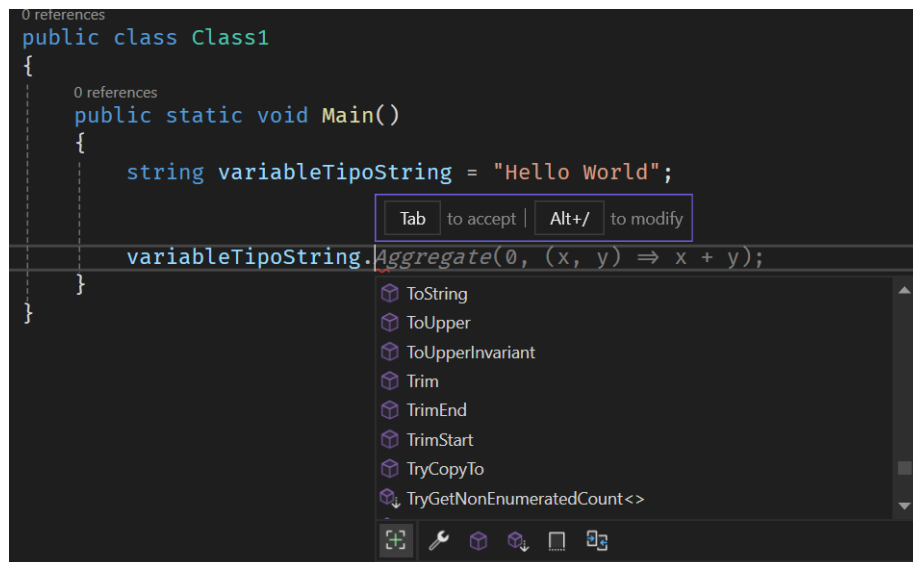


Figure 1.1: Ejemplo de autocompletado en el IDE según el tipo de dato

1.1.2 Tipo de dato simple y compuesto

Un *tipo de dato simple* es un tipo de dato que representa un único valor. Los tipos de datos simples son los tipos de datos básicos que se utilizan para representar valores individuales. No tiene sentido práctico separarlos en partes más pequeñas.

Un *tipo de dato compuesto* es un tipo de dato que representa una colección de valores. Los tipos de datos compuestos se utilizan para representar estructuras de datos más complejas que contienen múltiples valores de otros tipos de datos. Por ejemplo, un tipo de dato Cliente, puede contener los datos de nombre, edad, dirección, etc. **Los objetos se consideran tipos de datos compuestos.**

1.1.3 Tipo referencia y tipo valor

Un *tipo de referencia* es un tipo de dato que almacena una referencia a un ubicación en memoria. Los tipos de referencia se utilizan para acceder memoria que puede ser compartida y modificada por múltiples partes de un programa. Por ejemplo, en Python, las listas y los diccionarios son tipos de referencia. **Los objetos en la mayoría de lenguajes orientados a objetos también son tipos de referencia.**

Un *tipo valor* es un tipo de dato que almacena un valor directamente en la

memoria. Los tipos de valor se utilizan para representar valores que no pueden ser compartidos ni modificados por múltiples partes de un programa. Por ejemplo, en *Python*, los enteros y los flotantes son tipos de valor.

Usualmente los tipos de datos simples son tipos de valor, mientras que los tipos de datos compuestos son tipos de referencia. Puede leer más para el caso de *C#* aquí.

1.1.4 Tipos de Datos Abstractos

Un *tipo de dato abstracto (TDA)* es un modelo conceptual que define un conjunto de valores y un conjunto de operaciones que se pueden realizar con esos valores. Los TDAs son una forma de abstracción que permite a los programadores trabajar con datos de manera más abstracta y genérica.

Un TDA especifica una interfaz que define las operaciones que se pueden realizar con los datos, **pero no especifica cómo se implementan esas operaciones**. Esto permite a los programadores utilizar los TDAs sin tener que preocuparse por los detalles de implementación subyacentes. Por tanto, se puede decir que un TDA tiene una vista lógica y una vista física o de implementación.

```
classDiagram
class List {
    AddFirst(element: int)
    AddLast()
    DeleteFirst()
    DeleteLast()
    Find()
    Clear()
}
<<Interface>> List

class ArrayList {
    internalArray: int[];
}

class LinkedList {
    First: Node;
}

ArrayList ..|> List
LinkedList ..|> List
```

En el diagrama anterior, se ilustra como un TDA Lista, puede ser implementado mediante Nodos con memoria dinámica o mediante un arreglo con memoria estática. El API expuesto por el tipo Lista, no debe dar detalles de cómo se implementa internamente.

Una estructura de datos se puede entender como la implementación de TDA. En Programación Orientada a Objetos, **un TDA + implementación forman una clase**. Algunos lenguajes permiten definir *interfaces* que son un TDA puro.

1.2 Conceptos esenciales de programación orientada a objetos (POO)

POO es un paradigma de programación que modela conceptos del mundo real como *objetos* que tienen atributos y comportamientos. Los objetos son *ciudadanos de primera clase* en la programación orientada a objetos (POO), lo que significa que pueden ser manipulados y pasados como argumentos a funciones. Construir software orientado a objetos implica definir clases que representan tipos de objetos y crear instancias de esas clases (objetos) para interactuar entre sí.

Puede leer más sobre otros paradigmas de programación aquí.

1.2.1 Definición de Objeto

Estructura de datos que agrupa atributos (datos) y métodos (funciones) que operan sobre esos datos. Los objetos son instancias de clases, que definen la estructura (definida mediante los *atributos*) y comportamiento de los objetos (definido mediante los *métodos*).

La **interfaz** de un objeto se define por los métodos y atributos **públicos** que son accesibles desde fuera del objeto. Es decir, por otros objetos que usen el objeto en cuestión.

La mayoría de lenguajes de programación orientados a objetos, requieren la definición de clases para crear objetos.

1.2.2 Definición de Clase

Las clases son *plantillas que definen la estructura y comportamiento de los objetos*. Por ejemplo, la clase `Televisor` en C# se puede definir de la siguiente manera:

```
class Televisor
{
    // Atributos
    string marca;
    int pulgadas;
    bool encendido;

    // Métodos
    void Encender()
    {
        encendido = true;
    }
}
```

```

    }

    void Apagar()
    {
        encendido = false;
    }
}

```

Para generar objetos a partir de la clase Televisor, se utilizan las siguientes instrucciones:

```

Televisor samsung = new Televisor();
Televisor lg = new Televisor();
lg.Encender();
Console.WriteLine(samsung.encendido); // Imprime false

```

Un objeto opera sobre sus propios datos y no afecta la memoria de otros objetos diferentes. **Cada objeto es un bloque de memoria independiente que contiene sus propios datos y métodos.**

Clase vs Objeto

La clase define una plantilla, plano o molde para crear objetos basados en estos. Es decir, a partir de una clase se crean o instancian objetos

Definir clases es una forma de extender el lenguaje de programación, definiendo nuevos tipos de datos compuestos.

1.3 Principios de programación orientada a objetos

Son los conceptos fundamentales que rigen la programación orientada a objetos, la base para entender cómo se estructura y cómo se trabaja con este paradigma. Son los elementos distintivos que diferencian la programación orientada a objetos de otros paradigmas.

Los principios de POO son los siguientes: - Abstracción - Encapsulamiento - Herencia - Polimorfismo

1.3.1 Abstracción

Es la principal característica de POO. Permite modelar el problema por resolver en términos de objetos de alto nivel que ocultan los detalles de su implementación. La abstracción se logra a través de la creación de clases y objetos que representan entidades del mundo real. Por ejemplo, una clase **Persona** puede representar a una persona en el mundo real, con atributos como nombre, edad, etc., y métodos que permiten interactuar con la persona.

Al interactuar con objetos, pensamos en términos de su comportamiento y atributos en vez de variables y procedimientos.

Piezas de LEGO vs Plasticina

La abstracción permite que los objetos se comporten como piezas de LEGO que tienen una interfaz específica para interactuar con ellos, pero no necesitamos saber cómo están contruidos internamente. En cambio, al trabajar con plasticina, no hay una clara separación, sino que todo forma parte de un todo.

Aunque algunos lenguajes tienen la palabra reservada *Abstract*, esto no tiene relación con la abstracción como principio de POO.

1.3.2 Encapsulamiento

Los objetos son módulos autocontenidos que asocian código con sus datos. Los datos dentro de los objetos pueden o no exponerse según el programador lo decida. El **encapsulamiento** permite ocultar los detalles de implementación de un objeto y exponer solo la interfaz necesaria para interactuar con él.

Por ejemplo, en el código siguiente, los objetos de tipo *BankAccount*, ocultan los detalles de implementación de los métodos *Deposit* y *Withdraw*, y exponen solo la interfaz necesaria para interactuar con ellos. Ningún objeto externo a *BankAccount* puede acceder directamente a los atributos *accountNumber* y *balance*.

Los lenguajes orientados a objetos permiten establecer el nivel de visibilidad que tiene un atributo o método con respecto a otros objetos o clases. Por ejemplo, C# soporta muchos modificadores de acceso, entre los que se incluyen:

- **public**: Accesible desde cualquier parte del código.
- **private**: Accesible solo desde la misma clase.
- **protected**: Accesible desde la misma clase y sus clases derivadas.

Por ejemplo, considere la siguiente clase *Persona*:

```
public class Persona
{
    private string nombre;

    public string GetNombre()
    {
        return nombre;
    }

    public void SetNombre(string nombre)
    {
        this.nombre = nombre;
    }
}
```

```
3 references
public class BankAccount
{
    private string accountNumber;
    private decimal balance;

    1 reference
    public BankAccount(string accountNumber)
    {
        this.accountNumber = accountNumber;
        this.balance = 0;
    }

    1 reference
    public decimal GetBalance()
    {
        // Retorna una copia del valor de balance
        return balance;
    }

    1 reference
    public void Deposit(decimal amount)
    {
        balance += amount;
    }

    1 reference
    public void Withdraw(decimal amount)
    {
        if (amount <= balance)
        {
            balance -= amount;
        }
        else
        {
            Console.WriteLine("Fondos insuficientes");
        }
    }
}

0 references
public class Program
{
    0 references
    public static void Main(string[] args)
    {
        BankAccount account = new BankAccount("1234567890");
        account.Deposit(1000);
        account.Withdraw(500);
        decimal balance = account.GetBalance();
        Console.WriteLine("Account balance: " + balance);

        account.balance = 1000000;
    }
}
```

readonly struct System.Decimal
Represents a decimal floating-point number.

Describe with Copilot

CS0122: 'BankAccount.balance' is inaccessible due to its protection level

Show potential fixes (Alt+Enter or Ctrl+.)

Figure 1.2: Ejemplo de encapsulamiento

```
    }
}
```

En este ejemplo, el atributo nombre está declarado como `private`, lo que significa que solo es accesible desde la misma clase. Sin embargo, se proporcionan métodos públicos `GetNombre()` y `SetNombre()` para acceder y modificar el valor de nombre de manera controlada.

Puede leer sobre los modificadores de acceso en C# aquí. Igualmente puede leer más sobre `get` y `set` en C# aquí.

1.3.3 Herencia

La herencia permite definir jerarquías de objetos con el objetivo de reutilizar código. Cada clase puede tener máximo una clase padre de la que hereda atributos y métodos.

La clase padre se llama *superclase* y la clase hija se llama *subclase*. La superclase provee comportamiento general, mientras que las subclases proveen comportamiento especializado.

La herencia define una relación de tipo “es un/a” entre la superclase y la subclase. Por ejemplo, si tenemos una clase `Vehículo` y una clase `Automóvil`, podemos decir que *un automóvil es un vehículo*.

```
// Definición de la clase base
public class Animal
{
    public string Nombre { get; set; }
    public int Edad { get; set; }

    public void Comer()
    {
        Console.WriteLine("El animal está comiendo");
    }
}

// Definición de una subclase que hereda de Animal
public class Perro : Animal
{
    public void Ladrar()
    {
        Console.WriteLine("El perro está ladrando");
    }
}

// Uso de la herencia en el programa principal
public class Program
```

```

{
    public static void Main(string[] args)
    {
        // Creación de una instancia de la clase base
        Animal animal = new Animal();
        animal.Nombre = "Animal";
        animal.Edad = 5;
        animal.Comer();

        // Creación de una instancia de la subclase
        // Nótese que Perro hereda de Animal
        // las propiedades como Nombre y Edad
        Perro perro = new Perro();
        perro.Nombre = "Firulais";
        perro.Edad = 3;
        perro.Comer();
        perro.Ladrar();
    }
}

```

1.3.4 Polimorfismo

Etimológicamente significa “*muchas formas*” y se refiere a la capacidad de un objeto de comportarse de diferentes maneras. Hay dos tipos: *run-time* y *compile time*.

1.3.4.1 Run-time (tiempo de ejecución)

Cuando una clase hija anula (override) un método public/protected de la clase padre. Por ejemplo:

```

public class Shape
{
    public virtual void Draw()
    {
        Console.WriteLine("Drawing a shape");
    }
}

public class Circle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing a circle");
    }
}

```

```
public class Square : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing a square");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        Shape shape1 = new Circle();
        Shape shape2 = new Square();

        shape1.Draw(); // Output: Drawing a circle
        shape2.Draw(); // Output: Drawing a square
    }
}
```

Cuando el código cliente llama el método, el método se “resuelve” en ese momento invocando el método correcto. Para encontrar el método correcto, el compilador busca en la clase real del objeto en tiempo de ejecución y en caso de no estar, sigue subiendo por la jerarquía de clases.

Polimorfismo permite tratar la clase hija como si fuera la padre. Cualquier atributo o método visible de la clase Padre se puede acceder a través de la Hija. Cualquier método o atributo que sea específico de la hija, no es accesible a través de la clase padre.

Algunos lenguajes permiten crear clases abstractas que son útiles para polimorfismo.

```
public abstract class Animal
{
    void Respirar()
    {
        Console.WriteLine("Respirando")
    }

    abstract PorDefinir(); // no tiene definición y fuerza a las hijas a implementarla
}

public class Perro : Animal
{
    void Comer()
    {
        Console.WriteLine("Comiendo")
    }
}
```

```

    }

    // implementa el método abstracto de la clase padre.
    // Si no lo hace, el compilador arroja un error.
    void PorDefinir()
    {
        Console.Write("Foo...")
    }
}

```

Algunos lenguajes también permiten crear *interfaces* las cuales proveen polimorfismo sin formar parte de una jerarquía.

```

public interface Alimentable
{
    void alimentar();
}

public class Persona : Alimentable {

    // Implementa el método de la interfaz. Si no
    // lo hace, el compilador arroja un error.
    void alimentar()
    {
        Console.Write("Comiendo")
    }
}

```

1.3.4.2 Compile-time

La habilidad para sobrecargar (overload) métodos, es decir, crear varios métodos con el mismo nombre pero diferentes parámetros.

```

public class TestClass
{
    void foo()
    {
    }

    void foo(int x)
    {
    }

    void foo(string s, int x)
    {
    }
}

```


Chapter 2

Estructuras de Datos Lineales

¿Qué es una estructura de datos?

Una estructura de datos es una forma particular de organizar datos en una computadora para que puedan ser utilizados de manera eficiente. Se utilizan para procesar, recuperar y almacenar datos. Existe una colección mínima de estructuras de datos comunes que son utilizadas en la mayoría de los lenguajes de programación.

Las estructuras de datos lineales son estructuras de datos cuyos elementos se organizan secuencialmente, uno tras de otro. Hay un solo nivel de elementos y se recorren en una sola pasada. Cada elemento de la estructura tiene un predecesor y un sucesor, excepto el primer y el último elemento.

2.1 Arreglos y Matrices

2.1.1 Arreglos

Los *arreglos* son colecciones de elementos del mismo tipo. Los elementos se almacenan en posiciones contiguas de memoria. Es la estructura de datos más básica y se utiliza en la mayoría de los lenguajes de programación.

Terminología básica de arreglos incluye:

- *Índice*: los elementos en un array se identifican por su índice (empieza en cero usualmente). Por ejemplo, el primer elemento de un array tiene índice 0, el segundo tiene índice 1, y así sucesivamente.
- *Longitud*: el número de elementos en un array. Por ejemplo, un array con 5 elementos tiene longitud 5 y sus índices van de 0 a 4.

- *Elemento*: un valor almacenado en una posición específica del array. Por ejemplo, el elemento en la posición 0 del array es el primer elemento del array.

Visualmente, un arreglo se puede representar con la imagen siguiente. Nótese que cada elemento tiene una posición de memoria contigua (en la imagen se muestran direcciones de memoria en decimal). Cada “espacio” del arreglo es del mismo tamaño según el tipo de dato que se almacene (por ejemplo, un `int` ocupa 4 bytes, por eso observe como las direcciones aumentan de cuatro en cuatro).

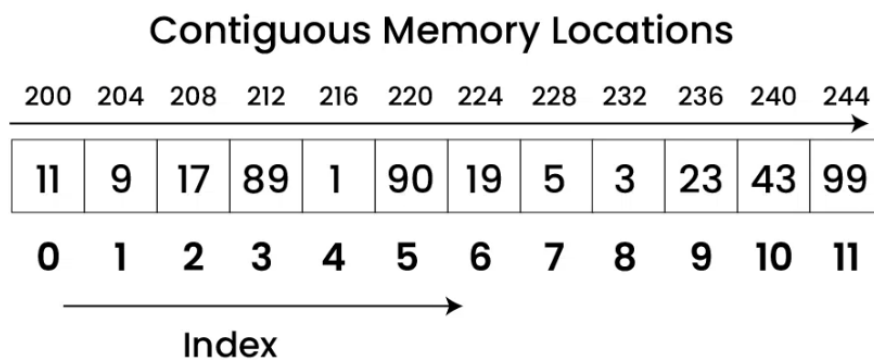


Figure 2.1: Representación de un arreglo en memoria

En C# una arreglo se declara de la siguiente manera:

```
// Arreglo de enteros. Cada posición es de 4 bytes
int[] arr;

// Arreglo de enteros. Cada posición es de 1 byte
char[] arr2;

// Arreglo de flotantes. Cada posición es de 4 bytes
float[] arr3;

// Ejemplo de inicialización en línea
int[] arr = { 1, 2, 3, 4, 5 };
char[] arr = { 'a', 'b', 'c', 'd', 'e' };
float[] arr = { 1.4f, 2.0f, 24f, 5.0f, 0.0f };

// Ejemplo de inicialización con tamaño
int[] arr = new int[5];

// Ejemplo de acceso a elementos
```

```
arr[0] = 1;  
int x = arr[0];
```

¿ArrayList?

Algunos lenguajes como C# proveen la clase ArrayList o similar. Esto corresponde a una implementación de una lista que se verá más adelante en este documento. No se debe confundir con un arreglo estático de posiciones contiguas

2.1.1.1 Ventajas y desventajas de los arreglos

Algunas de las ventajas de los arreglos son:

1. Tiene acceso aleatorio a los elementos. Es decir, se puede acceder a cualquier elemento en tiempo constante.
2. Es fácil de implementar y usar.
3. Cero overhead de memoria. No se necesita almacenar información adicional para mantener la estructura.

Por su parte, algunas de las desventajas de los arreglos son:

1. El tamaño del arreglo es fijo. No se puede cambiar el tamaño del arreglo una vez que se ha creado. Si se ocupa espacio adicional, se tiene que crear un nuevo arreglo y copiar los elementos del arreglo original al nuevo arreglo.
2. Es difícil de insertar y eliminar elementos. Se necesita mover todos los elementos que están después del elemento que se quiere insertar o eliminar.

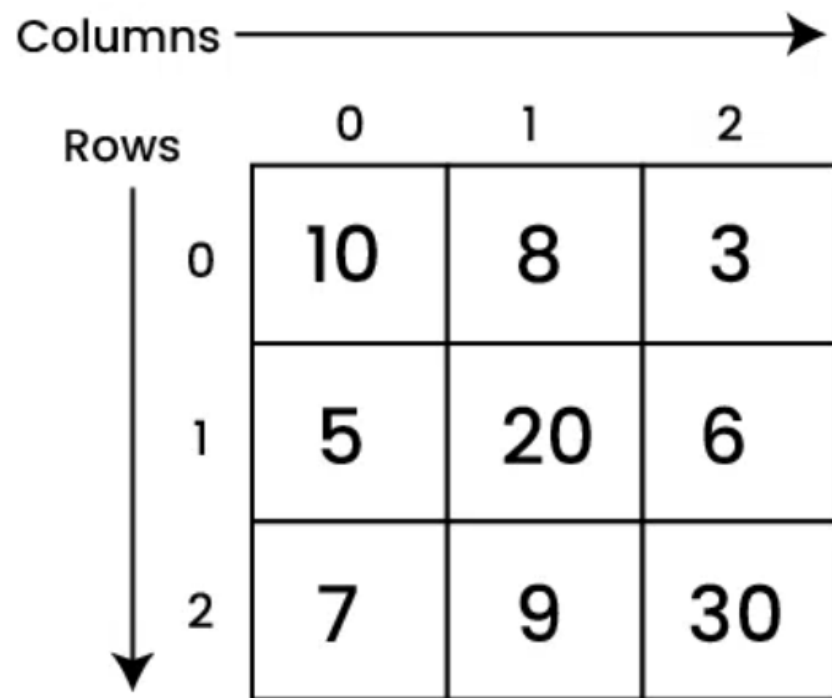
¿Acceso rápido?

Acceder un elemento de un arreglo, se reduce a realizar una operación de suma y una operación de acceso a memoria. Por ejemplo, si queremos acceder al elemento en la posición 3 de un arreglo de enteros, se realiza la siguiente operación: `arr + 3 * sizeof(int)`. Donde `arr` es la dirección de memoria del primer elemento del arreglo y `sizeof(int)` es el tamaño en bytes de un entero. Esta operación se realiza en tiempo constante, es decir, no importa el tamaño del arreglo, la operación siempre se realiza en el mismo tiempo.

2.1.2 Matrices

Son arreglos de dos dimensiones. Es decir, son arreglos de arreglos. Se utilizan para representar datos en forma de tabla. Por ejemplo, una matriz de 3x3 se puede representar gráficamente como:

Para declarar e inicializar una matriz en Java, se utiliza la siguiente sintaxis:



The diagram illustrates a 3x3 matrix. A horizontal arrow labeled "Columns" points to the right, and a vertical arrow labeled "Rows" points downwards. The columns are indexed 0, 1, and 2 from left to right. The rows are indexed 0, 1, and 2 from top to bottom. The matrix contains the following values:

	0	1	2
0	10	8	3
1	5	20	6
2	7	9	30

Figure 2.2: Representación de una matriz

```
int[] [] matriz = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};  
  
// También se puede declarar sin inicializar  
int[] [] matriz = new int[3][3];  
  
// Y luego inicializar  
matriz[0][0] = 1;  
matriz[0][1] = 2;
```

C# provee una forma más compacta de declarar e inicializar matrices:

```
int[,] matriz = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

Dado que son arreglos de arreglos, se puede acceder a los elementos de la matriz utilizando dos índices. Por ejemplo, para acceder al elemento en la fila 1 y columna 2 de la matriz anterior, se utiliza la siguiente sintaxis: `matriz[1][2]`.

¿Memoria contigua en las matrices?

Las matrices no necesariamente se almacenan en memoria contigua. Por ejemplo, en C# las matrices se almacenan en memoria contigua, pero en Java no. En Java, las matrices se almacenan como un arreglo de arreglos, es decir, un arreglo de referencias a otros arreglos. Por lo tanto, no se puede acceder a un elemento de la matriz utilizando una sola operación de suma y una operación de acceso a memoria. Se necesita realizar dos operaciones de acceso a memoria y una operación de suma. Esto se debe a que primero se necesita acceder a la referencia del arreglo que contiene el elemento y luego acceder al elemento dentro de ese arreglo.

Aunque el término matriz se utiliza comúnmente para referirse a arreglos de dos dimensiones, también se puede utilizar para referirse a arreglos de más de dos dimensiones. Por ejemplo, un arreglo de tres dimensiones se puede inicializar en Java de la siguiente manera:

```
int[] [] [] matriz3D = {  
    {  
        {1, 2, 3},  
        {4, 5, 6},  
        {7, 8, 9}  
    },  
    {  
        {1, 2, 3},  
        {4, 5, 6},  
        {7, 8, 9}  
    },  
    {  
        {1, 2, 3},  
        {4, 5, 6},  
        {7, 8, 9}  
    }  
};
```

```

        {10, 11, 12},
        {13, 14, 15},
        {16, 17, 18}
    },
    {
        {19, 20, 21},
        {22, 23, 24},
        {25, 26, 27}
    }
};

// También se puede declarar sin inicializar
int[] [] [] matriz3D = new int[3][3][3];

// Y luego inicializar
matriz3D[0][0][0] = 1;
matriz3D[0][0][1] = 2;

```

2.2 Listas

Son colecciones de elementos del mismo tipo. A diferencia de los arreglos, las listas no tienen un tamaño fijo. Se pueden agregar y eliminar elementos de la lista. El orden de inserción de los elementos se respeta.

2.2.1 Tipo de Dato Abstracto Lista

Las operaciones básicas que se pueden realizar en una lista son:

- **Obtener (get):** devuelve el elemento en una posición específica.
- **Agregar (add):** añade un elemento a la lista.
- **Eliminar (remove):** elimina un elemento de la lista.
- **Contiene (contains):** verifica si un elemento está en la lista.
- **Tamaño (size):** devuelve el número de elementos en la lista.

Pueden implementarse de muchas formas. Las más comunes son:

- **ArrayList:** Implementación mediante arreglos
- **SinglyLinkedList:** Implementación mediante listas simples enlazadas
- **DoubleLinkedList:** Implementación mediante listas doblemente enlazadas
- **CircularLinkedList:** Implementación mediante listas enlazadas circulares

2.2.2 Implementación mediante arreglos

```

public class ArrayList implements List {
    private int maxSize;
    private int currentSize;

```

```
private int[] storage;

public ArrayList(int size) {
    this.maxSize = size;
    this.currentSize = 0;
    this.storage = new int[size];
}

public void clear() {
    this.currentSize = 0;
}

public boolean isEmpty() {
    return this.currentSize == 0;
}

public int size() {
    return this.currentSize;
}

public boolean contains(int element) {
    for (int i = 0; i < this.maxSize; i++) {
        if (this.storage[i] == element) {
            return true;
        }
    }
    return false;
}

public boolean add(int element) {
    if (this.currentSize > this.maxSize) {
        return false;
    }
    this.storage[currentSize++] = element;
    return true;
}

public int remove(int element) {
    int i;
    for (i = 0; i < this.currentSize; i++) {
        if (this.storage[i] == element) {
            indexToRemove = i;
        }
    }
    if (i == this.maxSize) {
        throw new NoSuchElementException();
    }
}
```

```

    }
    for (int j = i; j < this.currentSize - 1; j++) {
        this.storage[j] = this.storage[j + 1];
    }
    this.currentSize--;
}
}

```

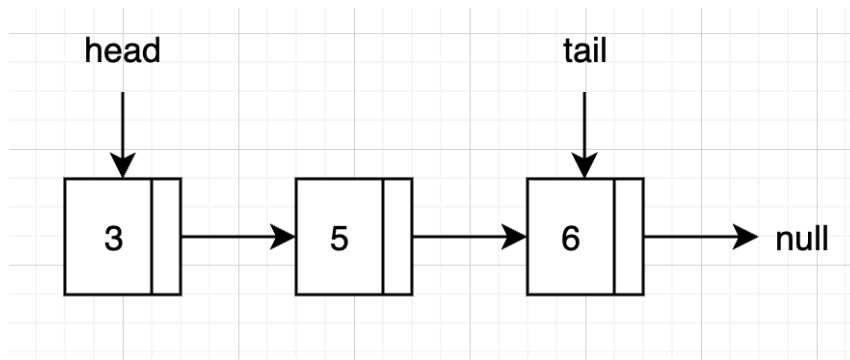
- Algunos lenguajes explicitamente proveen soporte para ArrayList
- **La principal ventaja** que proveen en vez de usar directamente es la abstracción de más alto nivel. El programador no tiene que lidiar con *shifts*, re-sizes u otros problemas de usar arreglos directamente
- Una estrategia que se puede utilizar para evitar lanzar una excepción al llegar al tamaño máximo, es crear un nuevo array de mayor tamaño y copiar los elementos del array actual. Pero si tendrá un *hit* de performance.

2.2.3 Implementación mediante listas simples enlazadas

Todos los enfoques utilizando listas enlazadas utilizan memoria dinámica en vez de un arreglo, es decir, asigna memoria en el *heap* cada vez que un elemento se agrega

Dado que crea la memoria *justo en el momento*, cada elemento de la lista está separado en la memoria (a diferencia de un array donde todos los elementos están en memoria contigua)

- Cada elemento de la lista es un *nodo*. Cada nodo tiene dos partes:
 - El **valor**: es el elemento relevante para el programador. El valor que solicitó registrar en la lista. Por ejemplo, `add(3)`, el 3 es el valor de interés para el programador
 - Una **referencia** al nodo siguiente que actúa como el elemento que encadena la lista.
- Visualmente, una lista enlazada se puede representar como:



- Como se puede notar, el último elemento apunta a *null* indicando el fin de la lista
- Es esencial llevar y mantener una referencia a la cabeza de la lista. Si la cabeza de la lista se pierde, se pierde toda la lista.
- Opcionalmente y para mejorar la eficiencia de la inserción al final, se puede llevar una referencia a la cola de la lista. Este tipo se conoce como *DoubleEndedLinkedList*.

2.2.3.1 Estructura general en Java

*// No es public, es una clase interna para no exponer la clase nodo a
// otras clases fuera del paquete*

```
class Node {
    int value;
    Node next;

    public Node(int value) {
        this.value = value;
        this.next = null;
    }
}

public class SinglyLinkedList implements List {
    private Node head;
    private int size;

    public SinglyLinkedList() {
        this.head = null;
        this.size = 0;
    }

    public void clear() {
        this.head = null;
        this.size = 0;
    }

    public boolean isEmpty() {
        return this.size == 0;
    }

    public int size() {
        return this.size;
    }

    public boolean contains(int element) {
        Node current = this.head;
```

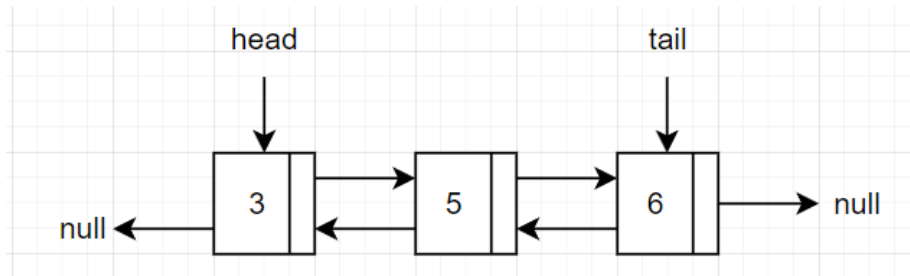
```
while (current != null) {
    if (current.value == element) {
        return true;
    }
    current = current.next;
}
return false;
}

public boolean add(int element) {
    Node newNode = new Node(element);
    if (this.head == null) {
        this.head = newNode;
    } else {
        Node current = this.head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
    this.size++;
    return true;
}

public int remove(int element) {
    if (this.head == null) {
        throw new NoSuchElementException();
    }
    if (this.head.value == element) {
        this.head = this.head.next;
        this.size--;
        return element;
    }
    Node current = this.head;
    while (current.next != null) {
        if (current.next.value == element) {
            current.next = current.next.next;
            this.size--;
            return element;
        }
        current = current.next;
    }
    throw new NoSuchElementException();
}
```

2.2.4 Implementación mediante lista doblemente enlazada

- La lista doblemente enlazada es similar a la lista simple enlazada, pero cada nodo tiene una referencia al nodo anterior y al siguiente
- Visualmente, una lista doblemente enlazada se puede representar como:



- La ventaja sobre la lista simple enlazada es que se puede recorrer la lista en ambas direcciones. La desventaja es que cada nodo tiene que mantener una referencia adicional al nodo anterior, lo que consume más memoria e implica mayor complejidad en la implementación.

2.2.4.1 Estructura general en Java

```

class Node {
    int value;
    Node next;
    Node prev;

    public Node(int value) {
        this.value = value;
        this.next = null;
        this.prev = null;
    }
}

public class DoubleLinkedList implements List {
    private Node head;
    private int size;

    public DoubleLinkedList() {
        this.head = null;
        this.size = 0;
    }

    public void clear() {
        this.head = null;
        this.size = 0;
    }
}

```

```
public boolean isEmpty() {
    return this.size == 0;
}

public int size() {
    return this.size;
}

public boolean contains(int element) {
    Node current = this.head;
    while (current != null) {
        if (current.value == element) {
            return true;
        }
        current = current.next;
    }
    return false;
}

public boolean add(int element) {
    Node newNode = new Node(element);
    if (this.head == null) {
        this.head = newNode;
    } else {
        Node current = this.head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
        newNode.prev = current;
    }
    this.size++;
    return true;
}

public int remove(int element) {
    if (this.head == null) {
        throw new NoSuchElementException();
    }
    if (this.head.value == element) {
        this.head = this.head.next;
        if (this.head != null) {
            this.head.prev = null;
        }
        this.size--;
    }
}
```

```

        return element;
    }
    Node current = this.head;
    while (current.next != null) {
        if (current.next.value == element) {
            current.next = current.next.next;
            if (current.next != null) {
                current.next.prev = current;
            }
            this.size--;
            return element;
        }
        current = current.next;
    }
    throw new NoSuchElementException();
}

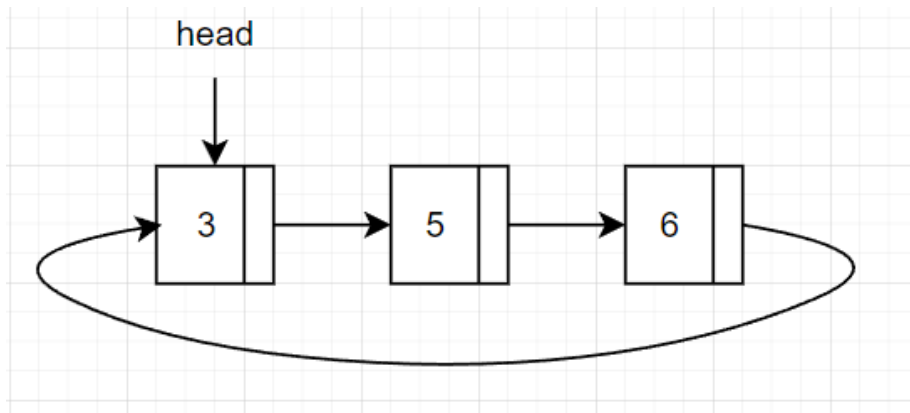
```

2.2.4.2 Aplicabilidad

- Cuando se necesita recorrer la lista en ambas direcciones, por ejemplo, en un editor de texto, donde se necesita recorrer el texto hacia adelante y hacia atrás o en un navegador web, donde se necesita recorrer el historial de navegación hacia adelante y hacia atrás.

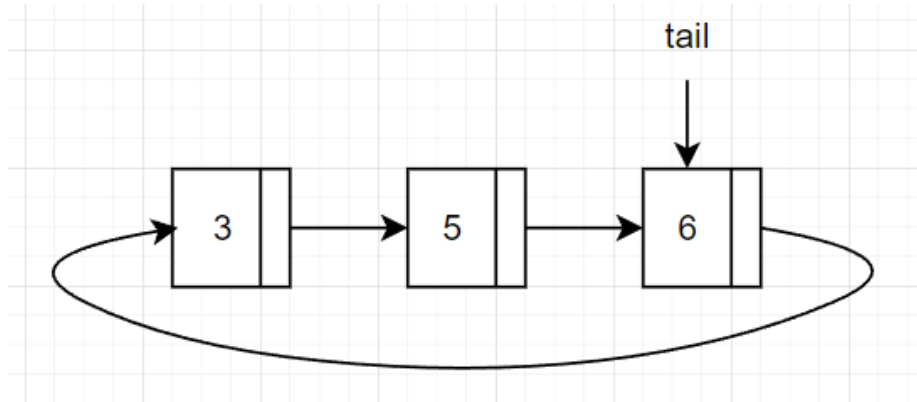
2.2.5 Implementación mediante lista enlazada circular

- La lista enlazada circular es similar a la lista simple enlazada, pero el último nodo apunta al primer nodo
- Visualmente, una lista enlazada circular se puede representar como:



- Usualmente se implementan como circular doblemente enlazada.

- Para mejorar las inserciones, en vez de mantener la referencia a *head* se utiliza una referencia a *tail* únicamente:



2.2.5.1 Estructura general en Java

```

class Node {
    int value;
    Node next;

    public Node(int value) {
        this.value = value;
        this.next = null;
    }
}

public class CircularSinglyLinkedList {
    private Node tail;
    private int size;

    public CircularSinglyLinkedList() {
        this.tail = null;
        this.size = 0;
    }

    public void clear() {
        this.tail = null;
        this.size = 0;
    }

    public boolean isEmpty() {
        return this.size == 0;
    }

    public int size() {

```

```
        return this.size;
    }

    public boolean contains(int element) {
        if (this.tail == null) {
            return false;
        }
        Node current = this.tail.next;
        while (current != this.tail) {
            if (current.value == element) {
                return true;
            }
            current = current.next;
        }
        return current.value == element;
    }

    public boolean add(int element) {
        Node newNode = new Node(element);
        if (this.tail == null) {
            newNode.next = newNode;
            this.tail = newNode;
        } else {
            newNode.next = this.tail.next;
            this.tail.next = newNode;
            this.tail = newNode;
        }
        this.size++;
        return true;
    }

    public int remove(int element) {
        if (this.tail == null) {
            throw new NoSuchElementException();
        }
        Node current = this.tail.next;
        Node prev = this.tail;
        while (current != this.tail) {
            if (current.value == element) {
                prev.next = current.next;
                this.size--;
                return element;
            }
            prev = current;
            current = current.next;
        }
    }
```

```

        if (current.value == element) {
            if (this.size == 1) {
                this.tail = null;
            } else {
                prev.next = current.next;
                if (current == this.tail) {
                    this.tail = prev;
                }
            }
            this.size--;
            return element;
        }
        throw new NoSuchElementException();
    }
}

```

2.2.5.2 Aplicabilidad

Cuando se necesita una lista que no tenga un final o un principio, por ejemplo, una lista de reproducción de música, donde la última canción apunta a la primera o una lista de tareas pendientes, donde la última tarea apunta a la primera.

2.2.6 Tabla comparativa de implementaciones

Implementación	Acceso	Insertar/Eliminar	Uso de Memoria
ArrayList	Rápido	Costoso	Siempre ocupa todo el espacio.
LinkedList	Lento	Rápido	No tiene overhead. Solo usa lo que ocupa. Tiene overhead.

2.3 Pilas

Una pila es una estructura de datos lineal que sigue el principio de LIFO (del inglés Last In, First Out), es decir, el último elemento en entrar es el primero en salir.

2.3.1 Tipo de dato abstracto Pila

La pila tiene dos operaciones básicas:

- **Apilar (push)**: añade un elemento a la pila.
- **Desapilar (pop)**: elimina el último elemento apilado.

- **Tope (top):** permite ver el elemento que está en la cima de la pila sin desapilarlo.

Before Push	After Push 3	After Push 5	After Pop	After Pop
		+---+		
		5		
	+---+	+---+	+---+	
+---+	3	3	3	+---+
+---+	+---+	+---+	+---+	+---+

```
public interface IStack {
    void push(int element);
    int pop();
    int top();
}
```

Al igual que en el caso de las listas, se pueden implementar pilas con arreglos o con listas enlazadas.

2.3.2 Implementación de pilas con arreglos

En la implementación de pilas con arreglos, se utiliza un arreglo unidimensional para almacenar los elementos de la pila.

```
public class ArrayStack implements IStack {
    private int[] stack;
    private int top;
    private int size;

    public ArrayStack(int size) {
        this.size = size;
        stack = new int[size];
        top = -1;
    }

    public void push(int element) {
        if (top == size - 1) {
            System.out.println("Stack Overflow");
        } else {
            stack[++top] = element;
        }
    }

    public int pop() {
        if (top == -1) {
            System.out.println("Stack Underflow");
            return -1;
        }
    }
}
```

```

    } else {
        return stack[top--];
    }
}

public int top() {
    if (top == -1) {
        System.out.println("Stack Underflow");
        return -1;
    } else {
        return stack[top];
    }
}
}

```

2.3.3 Implementación de pilas con listas enlazadas

```

public class LinkedListStack implements IStack {
    private Node top;

    public void push(int element) {
        Node newNode = new Node(element);
        newNode.next = top;
        top = newNode;
    }

    public int pop() {
        if (top == null) {
            System.out.println("Stack Underflow");
            return -1;
        } else {
            int element = top.data;
            top = top.next;
            return element;
        }
    }

    public int top() {
        if (top == null) {
            // Preguntar a los estudiantes, que es mejor, este enfoque o
            // lanzar excepciones como se vio en ejemplos pasados
            System.out.println("Stack Underflow");
            return -1;
        } else {
            return top.data;
        }
    }
}

```

```

    }

    // Otro enfoque para que la clase nodo solo pueda usarse dentro de
    // la clase Stack
    private class Node {
        int data;
        Node next;

        public Node(int data) {
            this.data = data;
        }
    }
}

```

2.4 Colas

Es una estructura de datos que sigue el principio de FIFO (del inglés First In, First Out), es decir, el primer elemento en entrar es el primero en salir. Respeta el orden de llegada de los elementos.

2.4.1 Tipo de dato abstracto Cola

```

public interface IQueue {
    void enqueue(int element);
    int dequeue();
    int front();
}

```

Cola vacía:

```
Frente --> Cola Vacía <-- Final
```

Agregando elementos 1, 2, 3, 4 y 5:

```
Frente --> [ 1 ] --> [ 2 ] --> [ 3 ] --> [ 4 ] --> [ 5 ] <-- Final
```

Quitando un elemento:

```
Frente --> [ 2 ] --> [ 3 ] --> [ 4 ] --> [ 5 ] <-- Final
```

Agregando el elemento 6:

```
Frente --> [ 2 ] --> [ 3 ] --> [ 4 ] --> [ 5 ] --> [ 6 ] <-- Final
```

Quitando dos elementos:

```
Frente --> [ 4 ] --> [ 5 ] --> [ 6 ] <-- Final
```

2.4.2 Implementación de colas con arreglos

```
public class ArrayQueue implements IQueue {
    private int[] queue;
    private int front;
    private int rear;
    private int size;

    public ArrayQueue(int size) {
        this.size = size;
        queue = new int[size];
        front = -1;
        rear = -1;
    }

    public void enqueue(int element) {
        if (rear == size - 1) {
            System.out.println("Queue Overflow");
        } else {
            if (front == -1) {
                front = 0;
            }
            queue[++rear] = element;
        }
    }

    public int dequeue() {
        if (front == -1) {
            System.out.println("Queue Underflow");
            return -1;
        } else {
            int element = queue[front];
            if (front == rear) {
                front = -1;
                rear = -1;
            } else {
                front++;
            }
            return element;
        }
    }

    public int front() {
        if (front == -1) {
            System.out.println("Queue Underflow");
            return -1;
        }
    }
}
```

```

        } else {
            return queue[front];
        }
    }
}

```

Esta implementación tiene un problema conocido como “drifting” que ocurre cuando se hacen muchas operaciones de inserción y eliminación. La cola se desplaza hacia la izquierda y se desperdicia espacio. Para solucionar este problema se puede implementar una cola circular.

```

public class CircularArrayQueue implements IQueue {
    private int[] queue;
    private int front;
    private int rear;
    private int size;

    public CircularArrayQueue(int size) {
        this.size = size;
        queue = new int[size];
        front = -1;
        rear = -1;
    }

    public void enqueue(int element) {
        if ((rear + 1) % size == front) {
            System.out.println("Queue Overflow");
        } else {
            if (front == -1) {
                front = 0;
            }
            rear = (rear + 1) % size;
            queue[rear] = element;
        }
    }

    public int dequeue() {
        if (front == -1) {
            System.out.println("Queue Underflow");
            return -1;
        } else {
            int element = queue[front];
            if (front == rear) {
                front = -1;
                rear = -1;
            } else {
                front = (front + 1) % size;
            }
        }
    }
}

```

```

        }
        return element;
    }
}

public int front() {
    if (front == -1) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        return queue[front];
    }
}
}

```

Con una cola circular, el frente y el final no necesariamente están al principio y al final del arreglo, respectivamente. En lugar de eso, el frente y el final se mueven a lo largo del arreglo. Cuando el final llega al final del arreglo, se mueve al principio del arreglo.

2.4.3 Implementación de colas con listas enlazadas

```

public class LinkedListQueue {
    private Node front;
    private Node rear;

    public LinkedListQueue() {
        front = null;
        rear = null;
    }

    public void enqueue(int element) {
        Node newNode = new Node(element);
        if (rear == null) {
            front = newNode;
            rear = newNode;
        } else {
            rear.next = newNode;
            rear = newNode;
        }
    }

    public int dequeue() {
        if (front == null) {
            System.out.println("Queue Underflow");
            return -1;
        }
    }
}

```

```

    } else {
        int element = front.data;
        front = front.next;
        if (front == null) {
            rear = null;
        }
        return element;
    }
}

public int front() {
    if (front == null) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        return front.data;
    }
}

private class Node {
    private int data;
    private Node next;

    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}
}

```

2.5 Colas de prioridad

Una cola de prioridad es una estructura de datos que almacena elementos en una cola, pero en lugar de seguir un orden de llegada, los elementos se organizan de acuerdo a su prioridad. Los elementos con mayor prioridad se desencolan antes que los elementos con menor prioridad.

2.5.1 Tipo de dato abstracto Cola de prioridad

```

public interface IPriorityQueue {
    void enqueue(int element, int priority);
    int dequeue();
    int front();
}

```

- **Encolar (enqueue):** Cuando un elemento se añade a la cosa, se mantiene

el orden de acuerdo a su prioridad, reorganizando los elementos si es necesario.

- **Desencolar (dequeue)**: Se elimina el elemento con mayor prioridad primero.
- **Tope (front)**: Permite ver el elemento con mayor prioridad sin desencolarlo.

2.5.2 Tipos de colas de prioridad

- Orden ascendente: el elemento con menor valor numérico tiene mayor prioridad.
- Orden descendente: el elemento con mayor valor numérico tiene mayor prioridad.

2.5.3 Implementación de colas de prioridad con listas enlazadas

Las colas de prioridad se pueden implementar de muchas maneras:

- Con arreglos
- Con listas enlazadas
- Con montículos (heaps)

Más adelante en el curso, veremos la implementación con heap. Por ahora, vamos a ver una implementación con listas enlazadas.

```
public class LinkedListPriorityQueue implements IPriorityQueue {
    private Node front;
    private Node rear;

    public LinkedListPriorityQueue() {
        front = null;
        rear = null;
    }

    public void enqueue(int element, int priority) {
        Node newNode = new Node(element, priority);
        if (front == null) {
            front = newNode;
            rear = newNode;
        } else if (priority < front.priority) {
            newNode.next = front;
            front = newNode;
        } else {
            Node current = front;
            while (current.next != null && current.next.priority <= priority) {
                current = current.next;
            }
        }
    }
}
```

```

    }
    newNode.next = current.next;
    current.next = newNode;
    if (newNode.next == null) {
        rear = newNode;
    }
}

}

public int dequeue() {
    if (front == null) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        int element = front.element;
        front = front.next;
        return element;
    }
}

}

public int front() {
    if (front == null) {
        System.out.println("Queue Underflow");
        return -1;
    } else {
        return front.element;
    }
}

}
}

```

2.6 Generics

Generics es aplicable a cualquier estructura de datos, no solo a las lineales.

Hasta ahora, los ejemplos vistos de las estructuras de datos, han sido únicamente para el tipo de dato `integer`. ¿Qué pasa si queremos implementar una estructura de datos para otro tipo de dato? Por ejemplo, si queremos implementar una pila para `String` o para `double`. En este caso, tendríamos que implementar una pila para cada tipo de dato que necesitemos. Esto no es eficiente y no es escalable.

Otra opción es utilizar el tipo de dato `Object` que es la superclase de todos los tipos de datos en Java/C#. Sin embargo, esto no es una solución óptima ya que se pierde el tipo de dato específico y se tendría que hacer un *casting* cada vez que se quiera utilizar el dato. Esto puede resultar en errores en tiempo de ejecución.

Java y muchos otros lenguajes, proveen el concepto de **generics** para solucionar este problema. Los **generics** permiten definir clases, interfaces y métodos con un tipo de dato que se especifica en el momento de la creación de la instancia.

```
public class LinkedList<T> {
    private Node<T> head;

    private static class Node<T> {
        private T data;
        private Node<T> next;

        public Node(T data) {
            this.data = data;
            this.next = null;
        }
    }

    public void add(T data) {
        Node<T> newNode = new Node<>(data);
        if (head == null) {
            head = newNode;
        } else {
            Node<T> current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }
    }

    // Other methods for LinkedList implementation...
}

/// Ejemplo de instanciación

public static void main(String[] args) {
    LinkedList<String> list = new LinkedList<>();
    list.add("Hello");
    list.add("World");
    list.add("Java");

    LinkedList<Integer> numbers = new LinkedList<>();
    numbers.add(1);
    numbers.add(2);

    LinkedList<Double> decimals = new LinkedList<>();
}
```

```
decimals.add(3.14);  
decimals.add(2.71);  
}
```

Una consideración importante al utilizar generics es al comparar los elementos en la estructura. Se puede aprovechar la interfaz `Comparable` de Java para forzar que los elementos de tipo `T` sean comparables entre sí.

```
public class LinkedList<T extends Comparable<T>> {  
    // ...  
  
    private class Node<T extends Comparable<T>> {  
        public int compareTo(T other) {  
            return this.data.compareTo(other);  
        }  
    }  
}
```

Para cualquier tipo de datos *personalizado*, se debe implementar la interfaz `Comparable` y definir las reglas de comparación que tengan sentido dentro del contexto de la aplicación.

- Clase `Persona`: comparar por cédula
- Clase `Estudiante`: comparar por número de carné

2.7 Referencias adicionales

- <https://www.geeksforgeeks.org/introduction-to-arrays-data-structure-and-algorithm-tutorials/>

Chapter 3

Estructuras de datos jerárquicas

Son estructuras de datos que permiten organizar los datos de manera jerárquica, es decir, en forma de árbol. Los datos se organizan de manera que exista un nodo raíz y cada nodo puede tener cero o más nodos hijos. Cada nodo hijo puede tener a su vez cero o más nodos hijos, y así sucesivamente.

Existen muchos tipos de árboles, cada uno con sus propias características y aplicaciones. Algunos de los árboles más comunes son:

- **Árboles binarios:** cada nodo tiene a lo sumo dos hijos.
 - **Árboles binarios de búsqueda:** Es un árbol binario con ciertas reglas que permiten realizar búsquedas eficientes.
 - **Árboles AVL:** Es un árbol binario de búsqueda balanceado.
 - **Árboles splay:** Es un árbol binario de búsqueda que reorganiza los nodos para que los nodos más utilizados estén cerca de la raíz.
 - **Árboles de expresiones:** Es un árbol que se utiliza para representar expresiones matemáticas.
 - **Árboles rojinegros:** Es un árbol binario de búsqueda balanceado.
 - **Heap:** Es un árbol binario que cumple con ciertas reglas y se utiliza para implementar colas de prioridad.
- **Árboles N-arios:** Cada nodo puede tener un número variable de hijos.
 - **Árboles B:** Es un árbol que permite almacenar grandes cantidades de datos en disco.
 - **Árboles B+:** Es una variante del árbol B que se utiliza en bases de datos y sistemas de archivos.
 - **Árboles trie:** Es un árbol que se utiliza para almacenar un conjunto de cadenas de caracteres.

- **Árboles de sufijos:** Es un árbol que se utiliza para almacenar todas las subcadenas de una cadena de caracteres.

A excepción de algunos árboles que puede implementarse con arreglos, la mayoría de los árboles se implementan con nodos enlazados, de forma similar a las *Listas*.

3.1 Terminología básica

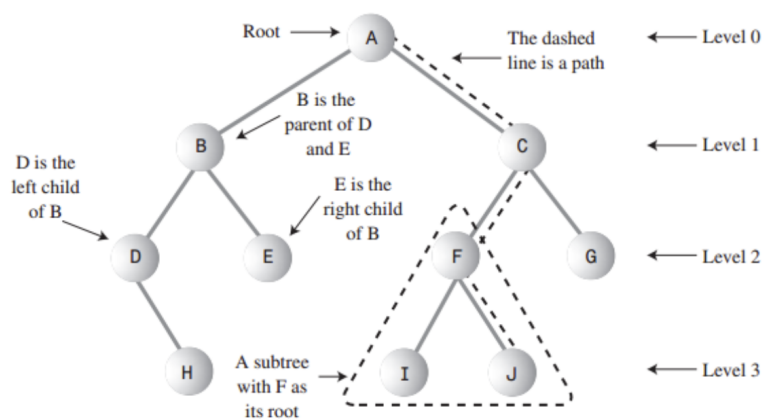


Figure 3.1: Árbol binario

3.2 Tipo de dato abstracto Árbol

Los árboles tienen las siguientes operaciones básicas:

- **Insertar (insert):** añade un nodo al árbol,
- **Eliminar (delete):** elimina un nodo del árbol,
- **Buscar (search):** busca un nodo en el árbol,
- **Recorrer (traverse):** recorre todos los nodos del árbol. Se puede realizar en distintos órdenes:
 - **Preorden:** primero se visita la raíz, luego el subárbol izquierdo y finalmente el subárbol derecho.
 - **Inorden:** primero se visita el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.
 - **Postorden:** primero se visita el subárbol izquierdo, luego el subárbol derecho y finalmente la raíz.
 - **Por niveles:** se recorren todos los nodos de un nivel antes de pasar al siguiente nivel.

3.3 Árboles Binarios de Búsqueda (BST)

Un BST es un árbol binario que cumple con la siguiente propiedad: para cada nodo, todos los nodos del subárbol izquierdo tienen un valor menor que el nodo y todos los nodos del subárbol derecho tienen un valor mayor que el nodo.

Los BST se pueden implementar mediante arrays o nodos enlazados. En este curso, BST se verán como nodos enlazados. Los *árboles Heap* que veremos más adelante, se implementarán con arrays.

3.3.1 Implementación

La estructura básica de un BST se puede implementar de la siguiente forma:

```
class TreeNode {
    int key;
    TreeNode left, right;

    public TreeNode(int item) {
        key = item;
        left = right = null;
    }
}

public class BinarySearchTree {
    TreeNode root;

    public BinarySearchTree() {
        root = null;
    }
}
```

3.3.2 Búsqueda de elementos

```
public boolean search(int key) {
    return searchRecursive(root, key);
}

private boolean searchRecursive(TreeNode root, int key) {
    if (root == null) {
        return false
    }
    if (root.key == key) {
        return true;
    }
    else if (key > root.key) {
        return searchRecursive(root.right, key);
    }
}
```

```

    } else {
        return searchRecursive(root.left, key);
    }
}

```

Dado que los árboles son una estructura jerárquica, la búsqueda se puede realizar de forma recursiva (preferida por simplicidad). De tal forma, la mayoría de los métodos van en parejas: *uno público que recibe los parámetros iniciales y otro privado que realiza la operación recursiva.*

3.3.3 Inserción de elementos

La inserción usa el mismo principio de búsqueda para encontrar el lugar donde se debe insertar el nuevo nodo. **Si el nodo ya existe, no se inserta.**

```

public void insert(int key) {
    root = insertRecursive(root, key);
}

private TreeNode insertRecursive(TreeNode root, int key) {
    if (root == null) {
        root = new TreeNode(key);
        return root;
    }

    if (key < root.key)
        root.left = insertRecursive(root.left, key);
    else if (key > root.key)
        root.right = insertRecursive(root.right, key);

    return root;
}

```

Note que el método `insert` re-assigna el valor de `root` al resultado de `insertRecursive`. Esto es necesario para que los cambios hechos en el árbol se reflejen en la raíz. Un error de principiante es creer que el siguiente código funciona:

```

public void insert(int key) {
    insertRecursive(root, key);
}

private void insertRecursive(TreeNode root, int key) {
    if (root == null) {
        root = new TreeNode(key);
        return;
    }
}

```

```

    if (key < root.key)
        insertRecursive(root.left, key);
    else if (key > root.key)
        insertRecursive(root.right, key);
}

```

No funciona puesto que **las referencias en Java se pasan por valor**. Esto significa que el valor de root no cambia en el método insert, por lo que el árbol no se modifica.

3.3.4 Eliminación de elementos

La eliminación en un BST considera varios casos:

- El nodo a eliminar es una hoja.
- El nodo a eliminar tiene un solo hijo.
- El nodo a eliminar tiene dos hijos.

Cuando el nodo es hoja, simplemente se elimina. **Cuando el nodo tiene un solo hijo**, se reemplaza el nodo por su hijo. **Cuando el nodo tiene dos hijos**, se reemplaza el nodo por el nodo más pequeño del subárbol derecho o por el nodo mayor del subárbol izquierdo (**solo se puede usar una de estas estrategias** en toda la implementación)

```

public void delete(int key) {
    root = deleteRecursive(root, key);
}

private TreeNode deleteRecursive(TreeNode root, int key) {
    if (root == null)
        return root;

    if (key < root.key)
        root.left = deleteRecursive(root.left, key);
    else if (key > root.key)
        root.right = deleteRecursive(root.right, key);
    else {
        if (root.left == null)
            return root.right;
        else if (root.right == null)
            return root.left;

        root.key = minValue(root.right);
        root.right = deleteRecursive(root.right, root.key);
    }
    return root;
}

```

```
private int minValue(TreeNode root) {  
    int minValue = root.key;  
    while (root.left != null) {  
        minValue = root.left.key;  
        root = root.left;  
    }  
    return minValue;  
}
```

3.3.5 Recorridos

Recorrer un árbol no solo es útil para imprimirlo, sino que también es útil para realizar operaciones en todos los nodos. Los recorridos más comunes son:

- Inorden (izquierda, raíz, derecha)
- Preorden (raíz, izquierda, derecha)
- Postorden (izquierda, derecha, raíz)

```
public void inOrder() {  
    inOrderRecursive(root);  
}  
  
private void inOrderRecursive(TreeNode root) {  
    if (root != null) {  
        inOrderRecursive(root.left);  
        System.out.print(root.key + " ");  
        inOrderRecursive(root.right);  
    }  
}  
  
public void preOrder() {  
    preOrderRecursive(root);  
}  
  
private void preOrderRecursive(TreeNode root) {  
    if (root != null) {  
        System.out.print(root.key + " ");  
        preOrderRecursive(root.left);  
        preOrderRecursive(root.right);  
    }  
}  
  
public void postOrder() {  
    postOrderRecursive(root);  
}
```

```

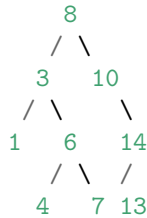
private void postOrderRecursive(TreeNode root) {
    if (root != null) {
        postOrderRecursive(root.left);
        postOrderRecursive(root.right);
        System.out.print(root.key + " ");
    }
}

```

3.3.5.1 In-orden

El recorrido in-orden de un árbol BST imprime los nodos en *orden ascendente*. Esto se debe a que el recorrido in-orden visita primero el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.

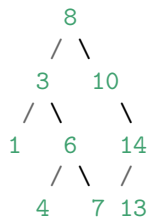
Por ejemplo, el siguiente árbol:



Se imprime como 1 3 4 6 7 8 10 13 14.

3.3.5.2 Pre-orden

El recorrido pre-orden de un árbol BST imprime la raíz antes que los subárboles. Esto se debe a que el recorrido pre-orden visita primero la raíz, luego el subárbol izquierdo y finalmente el subárbol derecho.

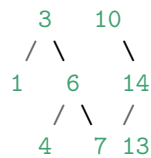


Se imprime como 8 3 1 6 4 7 10 14 13.

3.3.5.3 Post-orden

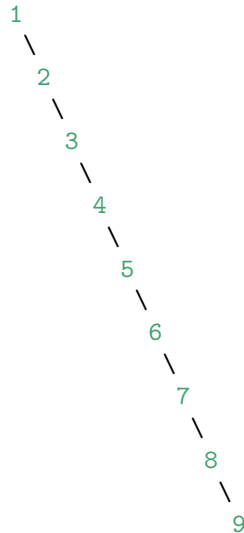
El recorrido post-orden de un árbol BST imprime los subárboles antes que la raíz. Esto se debe a que el recorrido post-orden visita primero el subárbol izquierdo, luego el subárbol derecho y finalmente la raíz.





Se imprime como 1 4 7 6 3 13 14 10 8.

- El problema de los BST de no ser balanceados * Los BST pueden degenerar en listas enlazadas si los elementos se insertan en orden. Esto puede ocurrir si los elementos se insertan en orden ascendente o descendente. En este caso, la búsqueda, inserción y eliminación se convierten en operaciones de tiempo lineal.



3.4 Árboles Heap (montículo)

- Un árbol *heap* es un árbol binario completo, es decir, todos los niveles del árbol están completamente llenos, excepto posiblemente el último nivel, que se llena de izquierda a derecha.
- Cumple con la propiedad de *heap*, que establece que el valor de cada nodo es mayor o igual que el valor de sus hijos. Si el valor del nodo es mayor que el de sus hijos, se trata de un árbol **heap máximo**, si el valor del nodo es menor que el de sus hijos, se trata de un árbol **heap mínimo**.
- Visualmente, un árbol *heap* se puede representar de la siguiente forma:
- Son una opción natural para implementar colas de prioridad.
- Son la estructura esencial para posteriormente implementar *heap sort* (lo

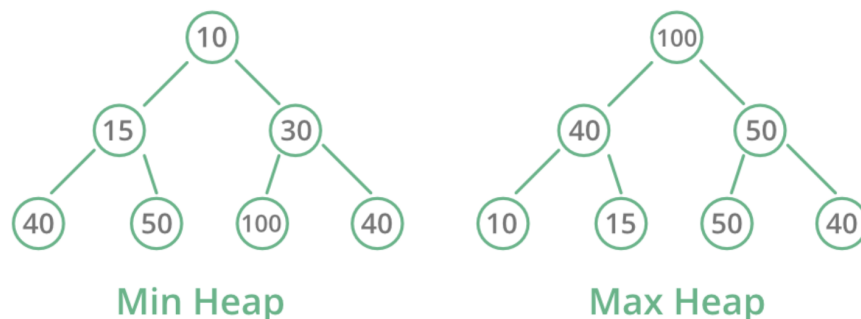


Figure 3.2: Árbol Heap

veremos más adelante en los algoritmos de ordenamiento).

3.4.1 Tipo de dato abstracto

Un heap tiene las siguientes operaciones:

- **Insertar (insert):** añade un elemento al heap y funciona de la siguiente manera:
 1. Añade el elemento al final del array.
 2. Compara el elemento con su padre y si es mayor, intercambia el elemento con su padre.
 3. Repite el paso 2 hasta que el elemento sea menor que su padre o llegue a la raíz del heap.
- **Eliminar (delete):** elimina el nodo raíz del heap,
- **Obtener (get):** permite ver el nodo raíz del heap sin eliminarlo.

```

public interface IHeap {
    void insert(int element);
    int delete();
    int get();
}

```

3.4.2 Implementación de árboles heap

Normalmente se implementan sobre un array. La raíz del árbol se almacena en la posición 0 del array, y los hijos de un nodo en la posición i se almacenan en las posiciones $2 * i + 1$ y $2 * i + 2$.

```

public class HeapTree {
    private int[] heap;
    private int size;
}

```

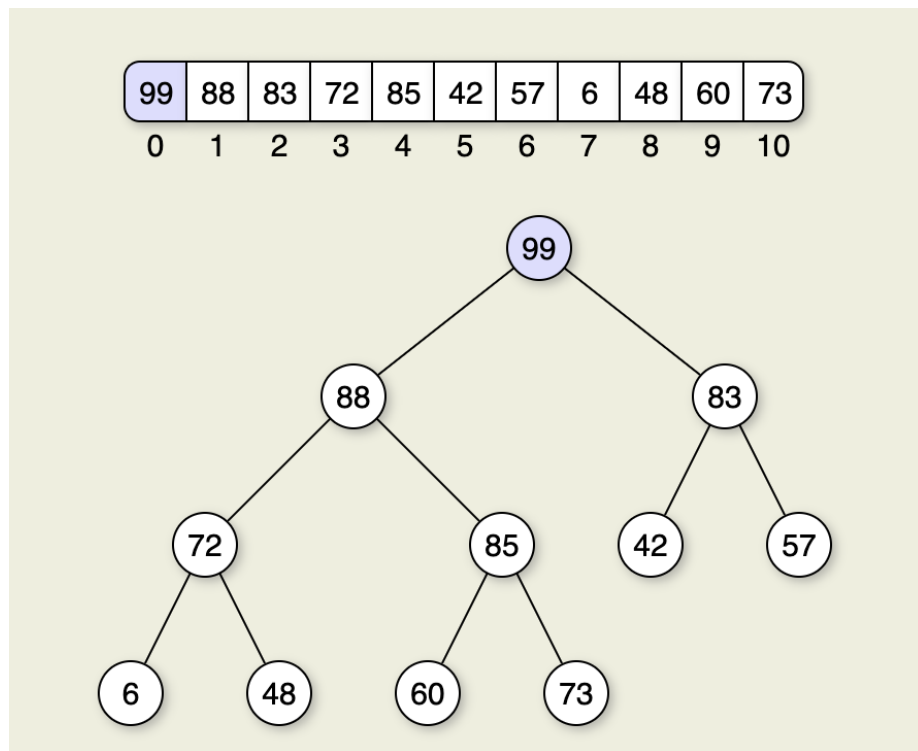


Figure 3.3: Árbol Heap

```
public HeapTree(int capacity) {
    heap = new int[capacity];
    size = 0;
}

public void insert(int element) {
    if (size == heap.length) {
        throw new IllegalStateException("Heap is full");
    }
    heap[size] = element;
    heapifyUp(size);
    size++;
}

public int delete() {
    if (size == 0) {
        throw new IllegalStateException("Heap is empty");
    }

    int root = heap[0];
    heap[0] = heap[size - 1];
    size--;
    heapifyDown(0);

    return root;
}

public int get() {
    if (size == 0) {
        throw new IllegalStateException("Heap is empty");
    }

    return heap[0];
}

private void heapifyUp(int index) {
    int parentIndex = (index - 1) / 2;

    while (index > 0 && heap[index] > heap[parentIndex]) {
        swap(index, parentIndex);
        index = parentIndex;
        parentIndex = (index - 1) / 2;
    }
}

private void heapifyDown(int index) {
```

```

    int leftChildIndex = 2 * index + 1;
    int rightChildIndex = 2 * index + 2;
    int largestIndex = index;

    if (leftChildIndex < size && heap[leftChildIndex] > heap[largestIndex]) {
        largestIndex = leftChildIndex;
    }

    if (rightChildIndex < size && heap[rightChildIndex] > heap[largestIndex]) {
        largestIndex = rightChildIndex;
    }

    if (largestIndex != index) {
        swap(index, largestIndex);
        heapifyDown(largestIndex);
    }
}

private void swap(int index1, int index2) {
    int temp = heap[index1];
    heap[index1] = heap[index2];
    heap[index2] = temp;
}
}

```

3.5 Árboles AVL

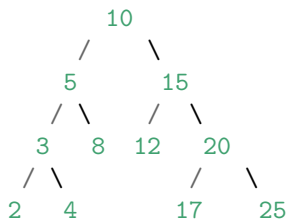
Los árboles AVL son árboles binarios de búsqueda **balanceados**. Fueron los primeros árboles auto-balanceados que se propusieron. Fueron introducidos por Adelson-Velsky y Landis en 1962.

Dado que se mantienen balanceados, las operaciones de búsqueda, inserción y eliminación tienen un tiempo de ejecución garantizado de $O(\log n)$ y no sufren de la degradación de rendimiento que sufren los árboles binarios de búsqueda no balanceados.

3.5.1 Características

- Es un árbol binario de búsqueda.
- Está balanceado en altura.
- El factor de balance de cada nodo es -1, 0 o 1.
- Las eliminaciones e inserciones pueden requerir balanceo a través de rotaciones.

El factor de balance de un nodo es la diferencia entre la altura del subárbol derecho y la altura del subárbol izquierdo.



Factores de balance de cada nodo:

Nodo	Factor de Balance
10	$4 - 4 = 0$
5	$2 - 3 = -1$
15	$3 - 2 = 1$
3	$2 - 2 = 0$
8	$1 - 1 = 0$
12	$1 - 1 = 0$
20	$2 - 2 = 0$
2	$0 - 0 = 0$
4	$0 - 0 = 0$
17	$1 - 1 = 0$
25	$1 - 1 = 0$

3.5.2 Ventajas

- Búsquedas rápidas
- Auto-balanceados

3.5.3 Desventajas

- Requieren más memoria que los árboles binarios de búsqueda no balanceados.
- Las operaciones de inserción y eliminación son más lentas que en los árboles binarios de búsqueda no balanceados.
- Las rotaciones pueden ser costosas.
- Difíciles de implementar.

3.5.4 Inserción

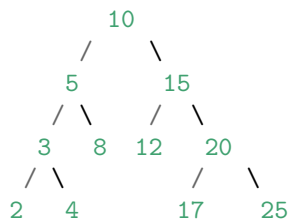
Para insertar un nodo w : - Se realiza una inserción normal de un árbol binario de búsqueda para el nodo w . - Iniciando en w , se recorre el camino de búsqueda hacia la raíz. Sea z el primero nodo no balanceado, y el hijo de z que está en el

camino de w a z y x el nieto de z que está en el camino de w a z . - Se rebalancea el árbol mediante rotaciones simples o dobles en el subárbol con raíz en z . Hay 4 posibles casos de rotaciones.

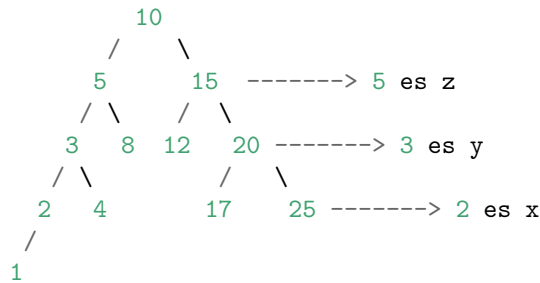
- ****Rotación derecha****: $_y_$ es el hijo izquierdo de $_z_$ y $_x_$ es el hijo izquierdo de $_y_$.
- ****Rotación Izquierda-derecha****: $_y_$ es el hijo izquierdo de $_z_$ y $_x_$ es el hijo derecho de $_y_$.
- ****Rotación Izquierda****: $_y_$ es el hijo derecho de $_z_$ y $_x_$ es el hijo derecho de $_y_$.
- ****Rotación Derecha-izquierda****: $_y_$ es el hijo derecho de $_z_$ y $_x_$ es el hijo izquierdo de $_y_$.

3.5.4.1 Ejemplo de rotación derecha

Dado el siguiente árbol AVL:



Se inserta el nodo 1:



El factor de balance del nodo 5 es 2, por lo que se debe rebalancear el árbol. El nodo 3 es el hijo de 5 que está en el camino de 1 a 5 y el nodo 2 es el nieto de 5 que está en el camino de 1 a 5. Estamos en el caso de rotación derecha.

Aplicando el siguiente código a z :

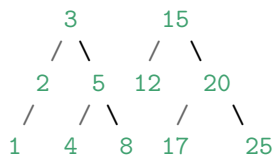
```

void rotateRight(Node node) {
    Node left = node.left;
    node.left = left.right;
    left.right = node;
    node = left;
}

```

Se obtiene:

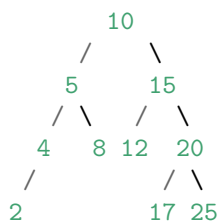




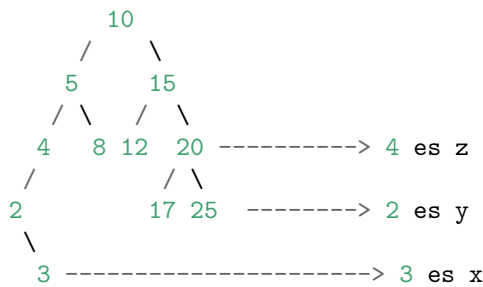
La rotación izquierda es la versión espejo de la rotación derecha.

3.5.4.2 Ejemplo de rotación izquierda-derecha

Dado el siguiente árbol AVL:



Se inserta el nodo 3:



El factor de balance del nodo 5 es 2, por lo que se debe rebalancear el árbol. El nodo 4 es el hijo de 5 que está en el camino de 3 a 5 y el nodo 2 es el nieto de 5 que está en el camino de 3 a 5. Estamos en el caso de rotación izquierda-derecha.

Aplicando el siguiente código a z:

```

void rotateLeftRight(Node node) {
    rotateLeft(node.left);
    rotateRight(node);
}

void rotateLeft(Node node) {
    Node right = node.right;
    node.right = right.left;
    right.left = node;
    node = right;
}

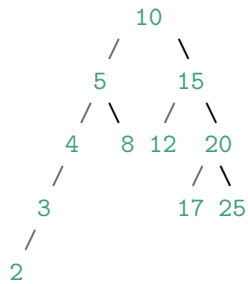
```

```

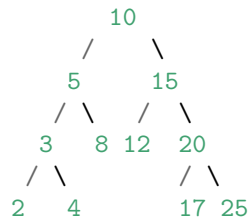
void rotateRight(Node node) {
    Node left = node.left;
    node.left = left.right;
    left.right = node;
    node = left;
}

```

Primero, se rota a la izquierda el nodo 2:



Luego, se rota a la derecha el nodo 4:



La rotación derecha-izquierda es la versión espejo de la rotación izquierda-derecha.

3.5.4.3 Implementación de inserción

```

class AVLNode {
    int key, height;
    AVLNode left, right;

    AVLNode(int key) {
        this.key = key;
        this.height = 1;
        this.left = this.right = null;
    }
}

public class AVLTree {

```

```
private AVLNode root;

// Get height of node
private int height(AVLNode node) {
    if (node == null)
        return 0;
    return node.height;
}

// Get balance factor of node
private int getBalance(AVLNode node) {
    if (node == null)
        return 0;
    return height(node.right) - height(node.left);
}

// Right rotate subtree rooted with y
private AVLNode rightRotate(AVLNode y) {
    AVLNode x = y.left;
    AVLNode T2 = x.right;

    // Perform rotation
    x.right = y;
    y.left = T2;

    // Update heights
    y.height = Math.max(height(y.left), height(y.right)) + 1;
    x.height = Math.max(height(x.left), height(x.right)) + 1;

    return x;
}

// Left rotate subtree rooted with x
private AVLNode leftRotate(AVLNode x) {
    AVLNode y = x.right;
    AVLNode T2 = y.left;

    // Perform rotation
    y.left = x;
    x.right = T2;

    // Update heights
    x.height = Math.max(height(x.left), height(x.right)) + 1;
    y.height = Math.max(height(y.left), height(y.right)) + 1;

    return y;
}
```

```

}

// Insert a key into the tree
public void insert(int key) {
    root = insertRecursive(root, key);
}

// Recursive function to insert a key into the tree
private AVLNode insertRecursive(AVLNode node, int key) {
    // Perform normal BST insertion
    if (node == null)
        return new AVLNode(key);

    if (key < node.key)
        node.left = insertRecursive(node.left, key);
    else if (key > node.key)
        node.right = insertRecursive(node.right, key);
    else // Duplicate keys not allowed
        return node;

    // Update height of this ancestor node
    node.height = 1 + Math.max(height(node.left), height(node.right));

    // Get the balance factor of this ancestor node
    int balance = getBalance(node);

    // If node becomes unbalanced, perform rotations
    // Left Left Case
    if (balance > 1 && key < node.left.key)
        return rightRotate(node);

    // Right Right Case
    if (balance < -1 && key > node.right.key)
        return leftRotate(node);

    // Left Right Case
    if (balance > 1 && key > node.left.key) {
        node.left = leftRotate(node.left);
        return rightRotate(node);
    }

    // Right Left Case
    if (balance < -1 && key < node.right.key) {
        node.right = rightRotate(node.right);
        return leftRotate(node);
    }
}

```

```

    return node;
}

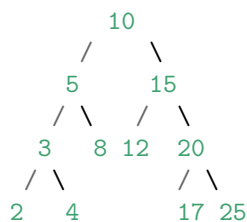
```

3.5.5 Eliminación

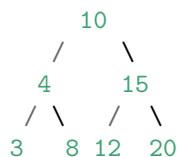
Para eliminar en un árbol AVL:

- Se realiza una eliminación normal de un árbol binario de búsqueda.
- Comenzando desde w , avanza hacia arriba y encuentra el primer nodo desequilibrado. Sea z el primer nodo desequilibrado, y el hijo de mayor altura de z , y x el hijo de mayor altura de y . Ten en cuenta que las definiciones de x e y son diferentes a las de la inserción aquí.
- Rebalancea el árbol realizando rotaciones apropiadas en el subárbol con raíz en z . Puede haber 4 casos posibles que deben ser manejados, ya que x , y y z pueden estar dispuestos de 4 formas diferentes. A continuación se presentan las 4 disposiciones posibles:
 - **Rotación derecha:** y es el hijo izquierdo de z y x es el hijo izquierdo de y .
 - **Rotación Izquierda-derecha:** y es el hijo izquierdo de z y x es el hijo derecho de y .
 - **Rotación Izquierda:** y es el hijo derecho de z y x es el hijo derecho de y .
 - **Rotación Derecha-izquierda:** y es el hijo derecho de z y x es el hijo izquierdo de y .

Dado el siguiente árbol, eliminemos 5:

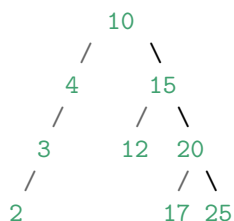


Se elimina el nodo 5 usando la lógica de eliminación de un árbol binario de búsqueda:

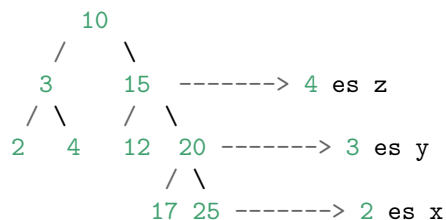




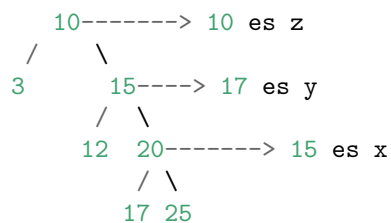
En este punto, no se cumple la propiedad de AVL. Se elimina 8:



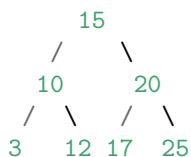
Se puede notar que 4 tiene un factor de balance -2, por lo que debe rebalancear usando rotación derecha en 4:



Se eliminan 2 y 4:



Estos nos dejan con un factor de balance en 10 de 2, por lo que se debe rebalancear usando rotación izquierda en 10:



3.5.6 Búsquedas y recorridos

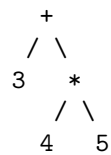
No hay cambios con respecto a BST

3.6 Árboles de expresión

Aunque no son técnicamente un TDA distinto, es relevante mencionarlos dado que su uso es muy común en la resolución de problemas de programación.

Son árboles binarios en los que las hojas son operandos y los nodos internos son operadores. Se utilizan para representar (y resolver) expresiones aritméticas o expresiones sintacticas de un lenguaje de programación en la etapa de compilación.

Por ejemplo, la expresión matemática $3 + (4 * 5)$ se puede representar con el siguiente árbol de expresión:



3.6.1 Conversión de arboles de expresión a notación infija

La notación infija es la forma tradicional de escribir expresiones matemáticas, en la que los operadores se escriben entre los operandos. Para generar la expresión, únicamente se requiere recorrer el árbol en inorden (izquierda, raíz, derecha).

```
private void inOrderRecursive(TreeNode root) {
    if (root != null) {
        inOrderRecursive(root.left);
        System.out.print(root.key + " ");
        inOrderRecursive(root.right);
    }
}
```

Árbol	Expresión
	(x + y) * (a - b)
	(x * (y - z)) * (a - f)
	(x * (y / -Z))
	(A + (B * - (C + D)))
	((A * (X + Y)) * C)

3.6.2 Conversión de expresión a árbol de expresión

Para convertir una expresión infija a un árbol de expresión, primero se debe convertir la expresión a notación postfija (postfix) y luego se debe recorrer la expresión postfija para construir el árbol.

3.6.3 Convertir expresión infija a postfija

Para convertir una expresión infija a postfija, se puede utilizar el algoritmo de Shunting Yard. Este algoritmo fue desarrollado por Edsger Dijkstra en 1961 y permite convertir una expresión infija a postfija en tiempo lineal.

El algoritmo utiliza dos estructuras de datos: una pila para almacenar los operadores y una cola para almacenar los operandos. El algoritmo recorre la expresión infija de izquierda a derecha y, dependiendo del tipo de token que se encuentre, realiza una de las siguientes acciones:

- Si el token es un operando, se añade a la cola.
- Si el token es un operador, se añade a la pila. Antes de añadirlo, se verifica si hay operadores en la pila con mayor precedencia. Si es así, se desapilan y se añaden a la cola.

Por ejemplo,

La cola (que contiene la expresión en postfijo) se utiliza como input para generar el árbol de expresión.

3.7 Árboles B

Los árboles B son una variante de los árboles N-arios que se utilizan para almacenar grandes cantidades de datos en disco. Los árboles B son muy utilizados en bases de datos y sistemas de archivos.

Descritos por Rudolf Bayer y Edward M. McCreight en 1972, los árboles B son árboles balanceados que se caracterizan por tener un número variable de hijos por nodo. Los árboles B son muy similares a los árboles binarios de búsqueda, pero con la diferencia de que cada nodo puede tener más de dos hijos.

Visualmente se pueden representar de la siguiente forma:

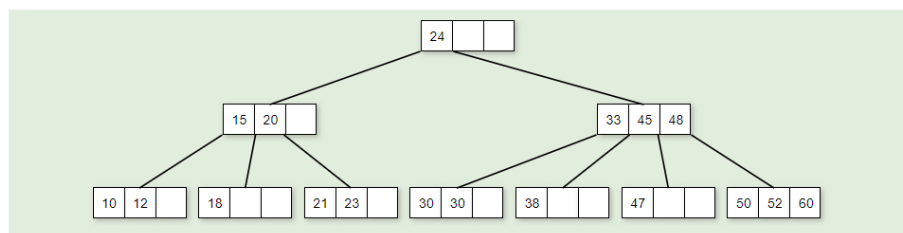


Figure 3.4: Árbol B

3.7.1 Características

Un árbol B de orden m , tiene las siguientes características:

- Cada nodo se compone de llaves y ramas. Las llaves están ordenadas y dividen las ramas en el orden esperado. Por ejemplo, entre las llaves 15 y 20, hay un nodo hijo, cuyas llaves serán mayores a 15 pero menores a 20.
- Todas las hojas están al mismo nivel.
- Se define con un orden m , que depende del tamaño del bloque del disco
- Cada nodo excepto la raíz **debe** contener al menos $m/2$ llaves. La raíz contiene un mínimo de una una llave.
- La inserción siempre ocurre en las hojas.
- Crecen o decrecen desde la raíz.

Pueden implementarse en memoria principal o en secundaria (propósito original). En este curso, nos enfocaremos en la implementación en memoria principal.

3.7.2 Estructura básica en Java

```
class BTreeNode {
    int order;
    int[] keys;
    int keyCount;
    BTreeNode[] branches;
}
```

3.7.3 Búsqueda

Es muy similar a la búsqueda de un elemento en un BST. Los pasos se pueden resumir de la siguiente manera para buscar una llave k :

1. Iniciando en la raíz, se compara k con las llaves del nodo. Si k se encuentra, se retorna el nodo o **true**.
2. Al ir comparando a k con cada una de las llaves, si se encuentra alguna llave mayor a k (o se llega al final de las llaves, osea k es mayor que todas), y hay una rama para dicha llave, se desciende por esa rama y se repite el proceso.
3. Si estamos en una rama, y no se encuentra k en las llaves, se return **null** o **false**

```
private Node Search(Node x, int key) {
    int i = 0;
    if (x == null)
        return x;
    for (i = 0; i < x.n; i++) {
        if (key < x.key[i]) {
            break;
        }
    }
    if (key == x.key[i]) {
        return x;
    }
}
```

```

    }
  }
  if (x.leaf) {
    return null;
  } else {
    return Search(x.child[i], key);
  }
}

```

3.7.4 Inserción

El árbol B crece hacia arriba desde la raíz. La inserción siempre ocurre en las hojas.

El proceso de inserción para una llave k sería:

- Se busca la llave en la raíz, entre las llaves del nodo.
- Dado que las llaves del nodo están ordenadas, se detiene si hay una llave mayor. Si tiene un hijo, se desciende a él y se repite el proceso.
- Si el nodo no tiene hijos:
 - Si el nodo no está lleno, se inserta en la posición correspondiente del arreglo de llaves
 - Si el nodo está lleno, se divide el nodo.

El proceso de división de un nodo n con $m-1$ llaves es el siguiente:

1. Se selecciona la llave mediana m y se sube al nodo padre.
2. Se crean dos nuevos nodos, $n1$ y $n2$.
3. Se distribuyen las llaves de n entre $n1$ y $n2$.
4. Se actualizan las ramas de $n1$ y $n2$.

El proceso de división se repite hasta llegar a la raíz.

Graficamente se puede ver de la siguiente manera (orden 5):

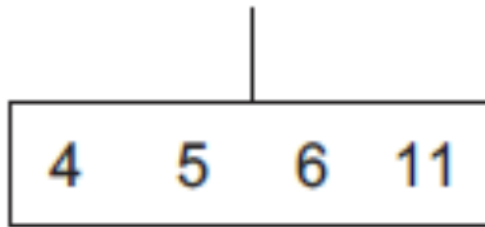


Figure 3.5: Árbol B

La raíz está llena. Al insertar la llave 8:

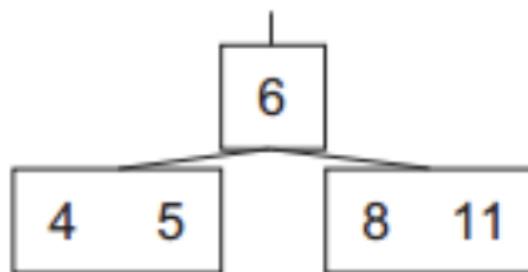


Figure 3.6: Árbol B

Varios elementos después:

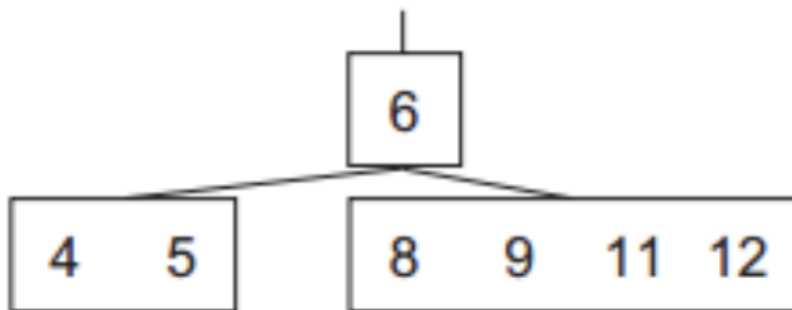


Figure 3.7: Árbol B

Varios elementos después...

3.7.5 Eliminación

Eliminar en un árbol B consiste de: 1. Encontrar el nodo que contiene la llave a eliminar. 2. Eliminar la llave del nodo. 3. Balancear el árbol para

3.7.5.1 Caso #1 - El nodo es hoja

En este caso, la llave por eliminar está en un nodo *hoja*. Hay dos sub-casos:

1.1 El nodo tiene más de $m-1$ llaves. En este caso, simplemente se elimina la llave.

Por ejemplo en este árbol de orden 3:

1.2 El nodo tiene $m-1$ llaves. En este caso, se *pide prestado* una llave de uno de los nodos hermanos. Primero se visita el hermano izquierdo. Si el hermano

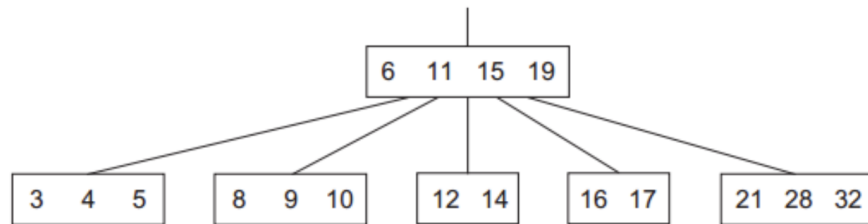


Figure 3.8: Árbol B

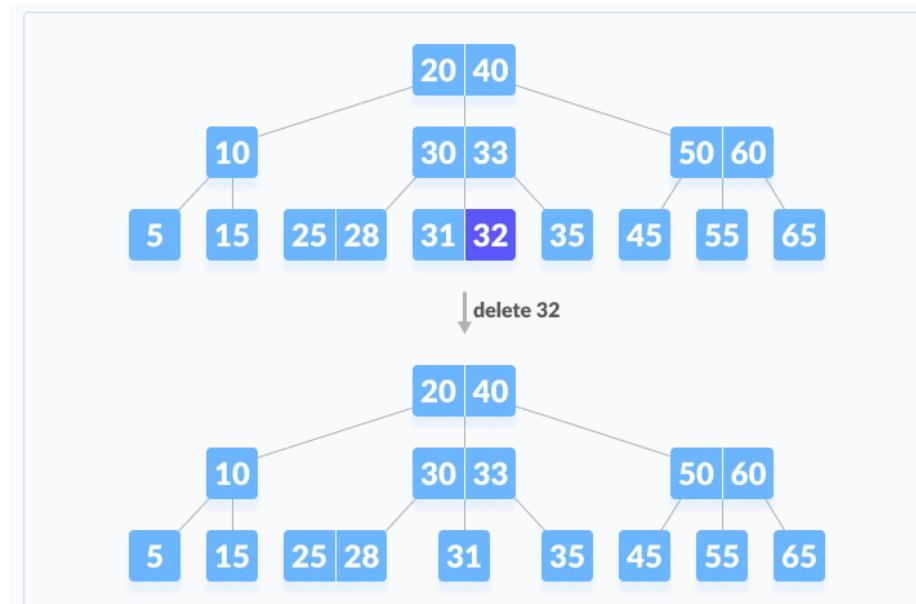


Figure 3.9: Árbol B

izquierdo tiene más de $m-1$ llaves, se toma la llave más grande del hermano izquierdo. Si no, se chequea el hermano derecho

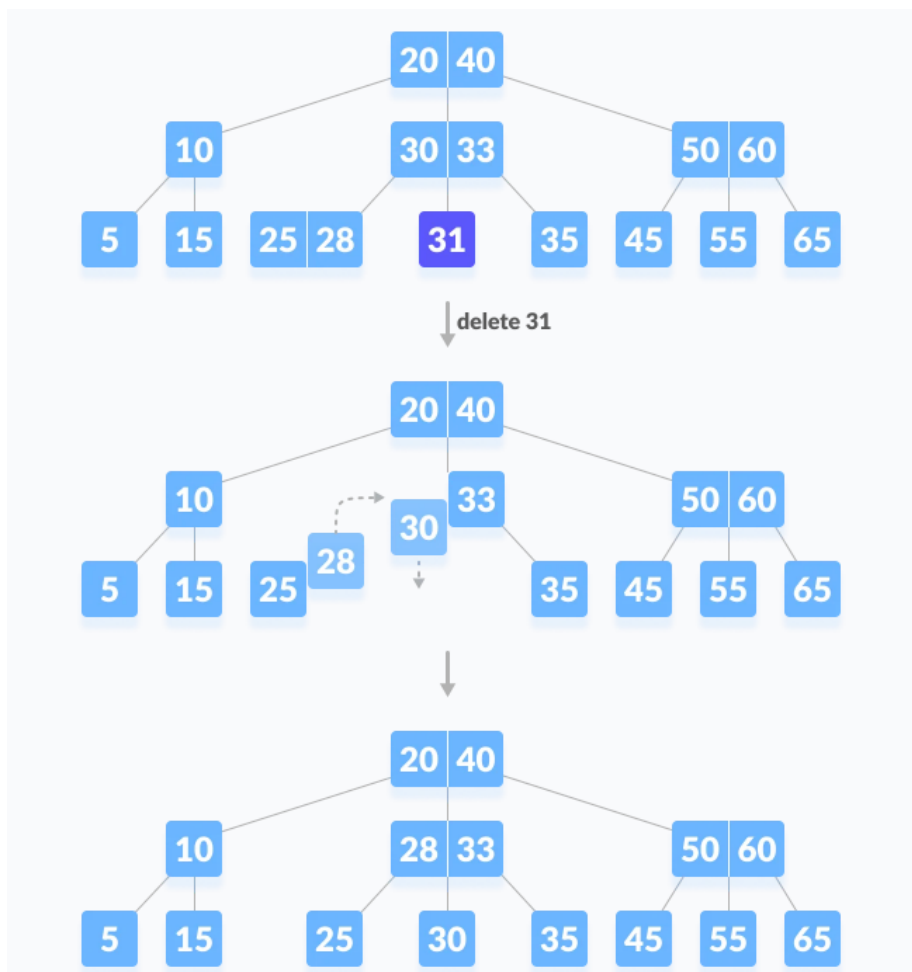


Figure 3.10: Árbol B

Si los dos hermanos tienen $m-1$ llaves, se fusionan los nodos y se elimina la llave del nodo padre. La mezcla de los nodos se hace mediante el nodo padre.

3.7.5.2 Caso #2 - El nodo es interno

Si la llave por eliminar está dentro de un nodo interno, los siguientes casos pueden ocurrir:

2.1 La llave eliminada se reemplaza por la llave inmediatamente mayor (o menor) del sub-árbol derecho (o izquierdo) del nodo, si siempre y cuando el sub-árbol

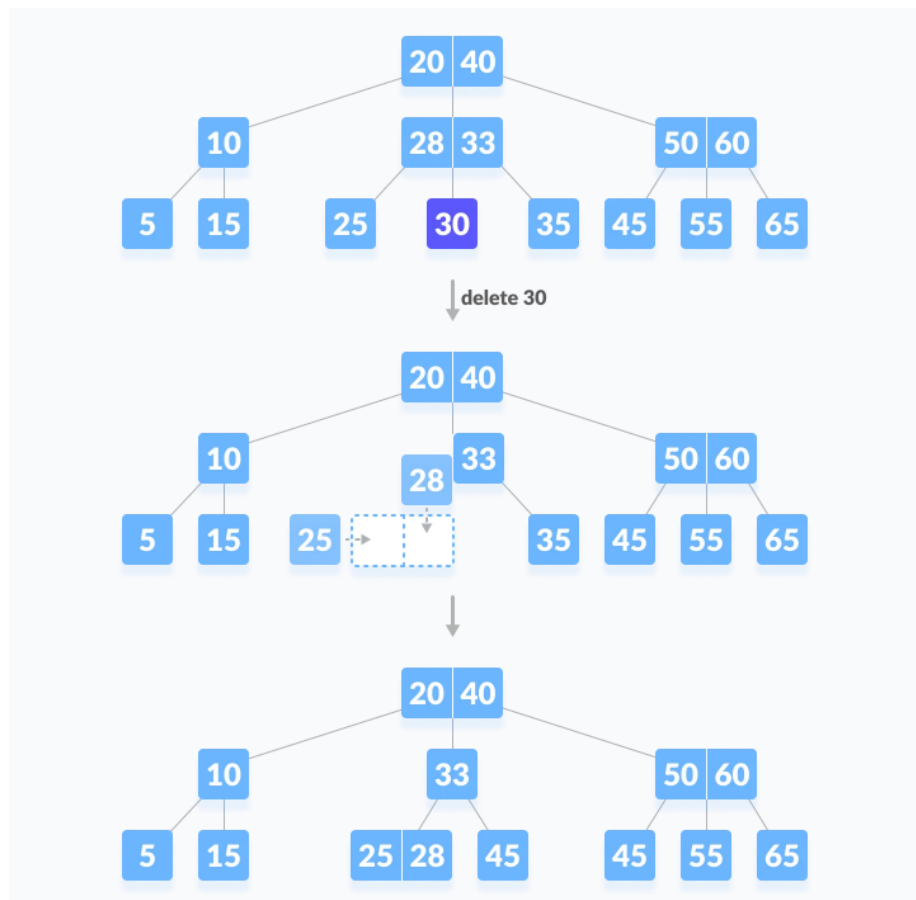


Figure 3.11: Árbol B

derecho (o izquierdo) más del mínimo de llaves.

2.2 Si ninguno de los hijos izquierdo o derecho tiene más de $m-1$ llaves, se fusionan los nodos y se elimina la llave del nodo padre.

3.7.5.3 Caso #3

La eliminación ocurre en un nodo interno. Si no se puede realizar el caso #2 (anterior), se unen los hijos junto con el padre.

3.8 Tries

Un Trie (del inglés *reTRIEval*) es una estructura de datos que permite almacenar un conjunto de cadenas de caracteres y realizar búsquedas de palabras en ellas. Los Tries son árboles de búsqueda n-arios que almacenan cadenas de caracteres, donde cada nodo del árbol representa un carácter. Los Tries son útiles para realizar búsquedas de palabras en un conjunto de cadenas de caracteres, como en diccionarios o en motores de búsqueda.

Se pronuncia como el inglés *try*.

Visualmente, un trie se puede ilustrar como:

Cada rama de un nodo corresponde a un carácter de la llave insertada. El último nodo de cada llave se conoce como *EndOfWord*. La raíz no tiene un valor asignado.

3.8.1 El TDA Trie

El TDA Trie tiene las siguientes operaciones básicas:

- **Insertar (insert)**: añade una cadena de caracteres al trie,
- **Buscar (search)**: busca una cadena de caracteres en el trie,
- **Eliminar (delete)**: elimina una cadena de caracteres del trie.

3.8.2 Estructura básica

```
class TrieNode
{
    TrieNode[] children = new TrieNode[ALPHABET_SIZE];
    boolean isEndOfWord;

    TrieNode(){
        isEndOfWord = false;
        for (int i = 0; i < ALPHABET_SIZE; i++)
            children[i] = null;
    }
}
```

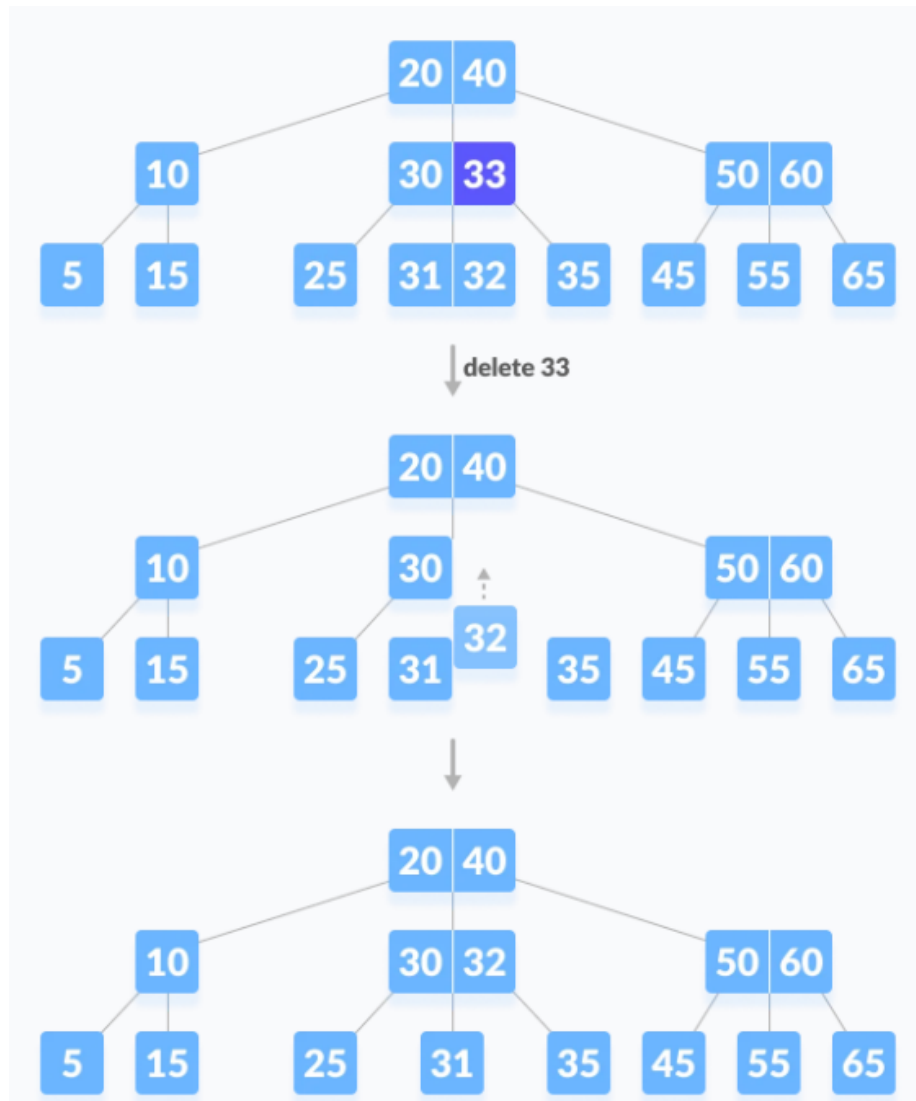


Figure 3.12: Árbol B

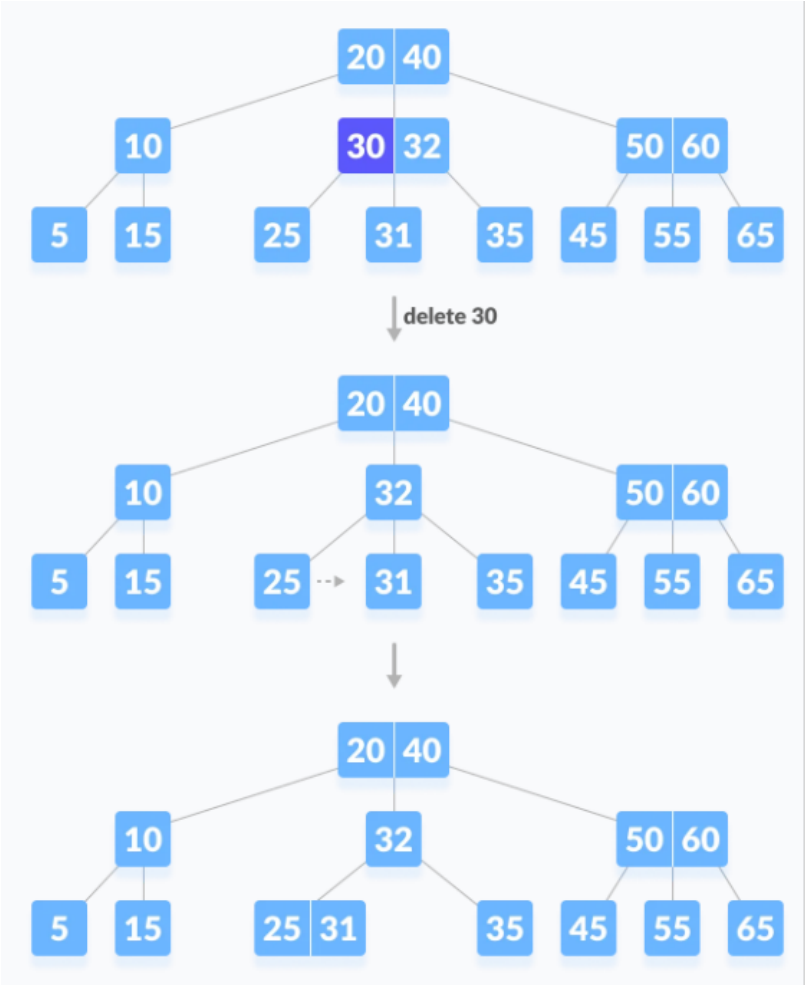


Figure 3.13: Árbol B

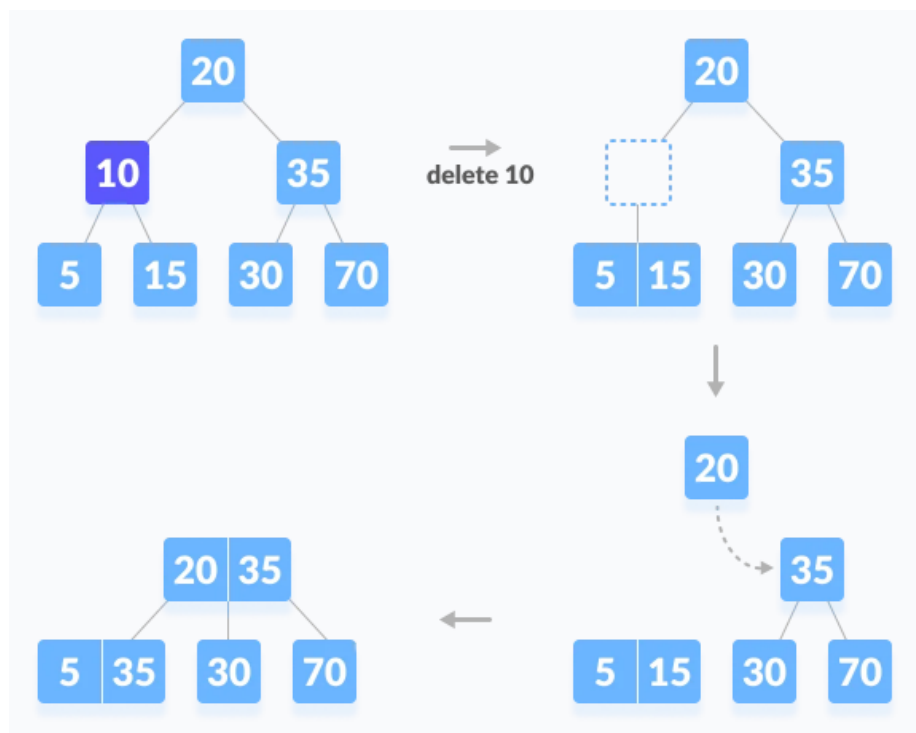


Figure 3.14: Árbol B

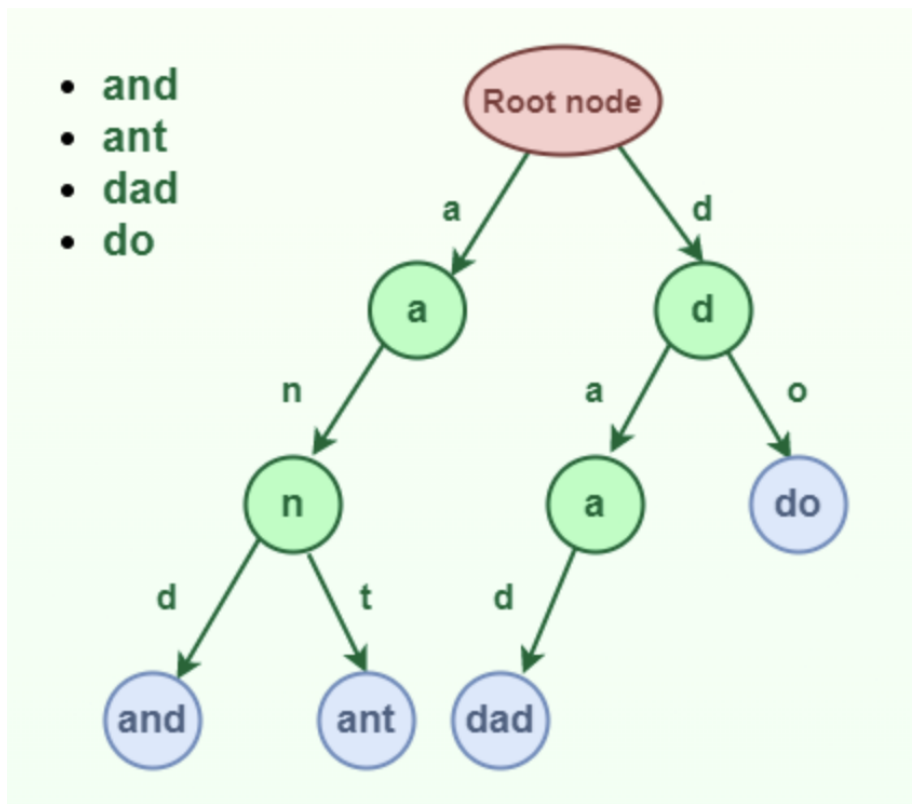


Figure 3.15: Trie

```

class Trie
{
    TrieNode root;
}

```

3.8.3 Inserción

Insertar una *llave* en un Trie es un proceso simple:

- Cada caracter de la llave se inserta como un nodo Trie individual.
- Los hijos de un nodo son un arreglo de punteros (o referencias) a los nodos del siguiente nivel del trie.
- El caracter de la llave actúa como un índice para el arreglo de hijos. Si la llave de entrada es nueva o una extensión de la llave existente, se construyen nodos no existentes de la llave y se marca el final de la palabra para el último nodo.
- Si la llave de entrada es un prefijo de la llave existente en el Trie, simplemente se marca el último nodo de la llave como el final de una palabra.

La longitud de la llave determina la profundidad del Trie.

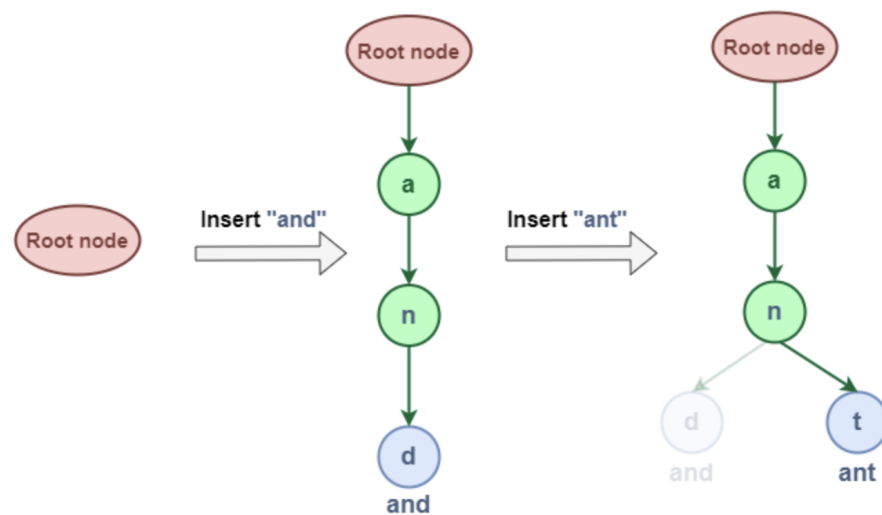


Figure 3.16: Trie

```

void insert(String key) {
    int level;
    int length = key.length();
    int index;
}

```

```

TrieNode pCrawl = root;

for (level = 0; level < length; level++) {
    index = key.charAt(level) - 'a';
    if (pCrawl.children[index] == null)
        pCrawl.children[index] = new TrieNode();

    pCrawl = pCrawl.children[index];
}

// mark last node as leaf
pCrawl.isEndOfWord = true;
}

```

3.8.4 Búsqueda

Buscar una llave en un Trie es similar a la operación de inserción. Sin embargo, se detiene si no hay más caracteres en la llave o si no hay más nodos en el Trie. La búsqueda puede terminar debido al final de una cadena o la falta de una llave en el Trie.

- En el primer caso, si el campo `isEndOfWord` del último nodo es verdadero, entonces la llave existe en el Trie.
- En el segundo caso, la búsqueda termina sin examinar todos los caracteres de la llave, ya que la llave no está presente en el Trie.

```

boolean search(String key) {
    int level;
    int length = key.length();
    int index;
    TrieNode pCrawl = root;

    for (level = 0; level < length; level++) {
        index = key.charAt(level) - 'a';

        if (pCrawl.children[index] == null)
            return false;

        pCrawl = pCrawl.children[index];
    }
    return (pCrawl.isEndOfWord);
}

```


Chapter 4

Algoritmos de búsqueda y ordenamiento

4.1 Algoritmos de búsqueda

Como lo indica su nombre, permiten buscar un elemento dentro de una colección de datos. La colección puede ser sobre arreglos o listas enlazadas, sin embargo, en este curso se enfocará en arreglos.

4.1.1 Búsqueda lineal

La búsqueda lineal es el método más sencillo para buscar un elemento en un arreglo (o una lista). Consiste en recorrer el arreglo desde el primer elemento hasta el último, comparando cada elemento con el valor que se busca.

Cuando el arreglo **no está ordenado**, la búsqueda lineal es la **única opción**. Sin embargo, cuando el arreglo está ordenado, existen algoritmos más eficientes para realizar la búsqueda, como la búsqueda binaria.

Dado el siguiente arreglo:

Indices	0	1	2	3	4	5	6	7	8	
Elementos	5	3	8	6	2	7	1	4	9	

Para buscar el número 7, se sigue el siguiente proceso:

Indices	0	1	2	3	4	5	6	7	8	
Elementos	5	3	8	6	2	7	1	4	9	
5	!=	7								
3	!=	7								
8	!=	7								
6	!=	7								

```

2 != 7           ^
7 == 7           ^

```

4.1.1.1 Implementación

```

public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}

```

4.1.2 Búsqueda binaria

Es un algoritmo de búsqueda que encuentra la posición de un valor en un arreglo ordenado. A diferencia de la búsqueda lineal, que recorre el arreglo desde el primer elemento hasta el último, la búsqueda binaria divide el arreglo en dos mitades y compara el valor buscado con el elemento en el medio.

- Si el valor buscado *es menor* que el elemento en el medio, la búsqueda continúa en la mitad **izquierda** del arreglo.
- Si el valor buscado *es mayor* que el elemento en el medio, la búsqueda continúa en la mitad **derecha** del arreglo.

Este proceso se repite hasta que el valor buscado sea encontrado o hasta que el subarreglo de búsqueda sea vacío.

Dado el siguiente arreglo:

Indices	0	1	2	3	4	5	6	7	8
Elementos	1	2	3	4	5	6	7	8	9

Para buscar el número 9:

Indices	0	1	2	3	4	5	6	7	8
Elementos	1	2	3	4	5	6	7	8	9


```

5 < 9           ^
               |-----|
7 < 9           ^
               |-----|
8 < 9           ^
               |-----|
9 == 9         ^

```

4.1.2.1 Implementación

Puede implementarse recursivamente o iterativamente. A continuación se muestra la implementación iterativa:

```
public static int binarySearch(int[] arr, int target) {  
    int left = 0;  
    int right = arr.length - 1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
  
        if (arr[mid] == target) {  
            return mid;  
        }  
  
        if (arr[mid] < target) {  
            left = mid + 1;  
        } else {  
            right = mid - 1;  
        }  
    }  
  
    return -1;  
}
```

4.1.3 Búsqueda Hash

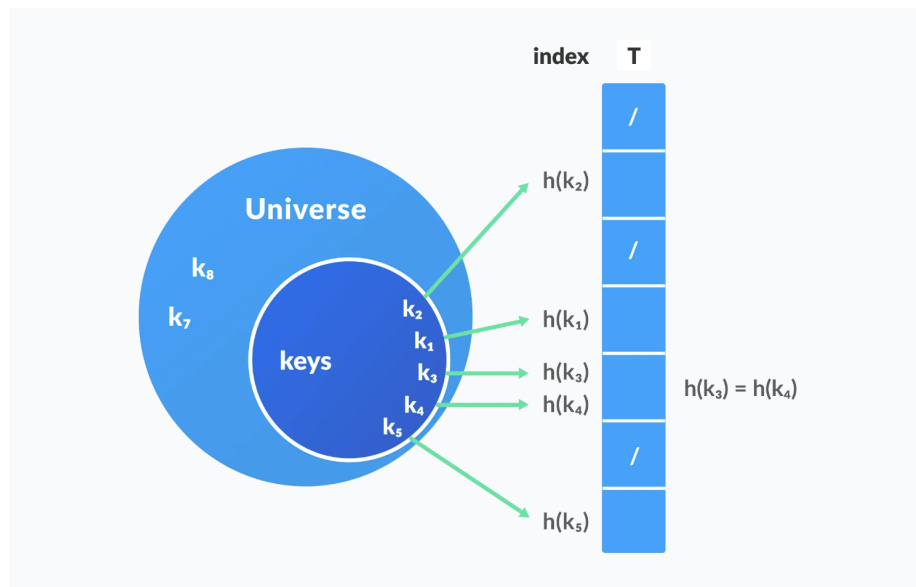
Para buscar en grandes colecciones de datos, no necesariamente ordenados, *hashing* (dispersión) provee una técnica para buscar de una forma más eficiente. La función de *hash* se utiliza para transformar una o más características de cada elemento del universo de búsqueda en un valor numérico que corresponde a un índice de un array. La búsqueda con *hash*, tiene un mejor rendimiento promedio que otros algoritmos de búsqueda.

4.1.3.1 Hash tables

Es una estructura de datos que almacena datos en pares *llave-valor*:

- *Llave*: llave única para identificar un valor
- *Valor*: dato asociado con la llave

La llave k se utiliza como entrada para una función de hashing $h(k)$ que genera un índice i donde el valor se almacenará dentro de la tabla. Visualmente se puede representar de esta forma:



Por ejemplo, suponga que se tiene un conjunto de datos que representan ciudadanos costarricenses, donde cada registro, contiene una cédula numérica que identifica cada registro.

Cédula	Nombre	Apellido	Dirección
123456789	Juan	Pérez	San José
987654321	María	Rodríguez	Heredia
456789123	Carlos	Sánchez	Alajuela

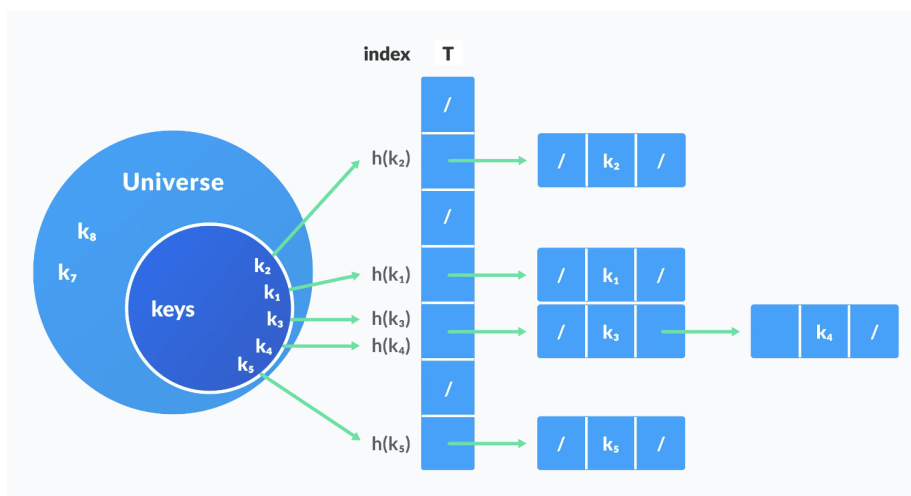
Para buscar un ciudadano en particular, se puede utilizar la cédula como llave y almacenar los datos en una tabla *hash*. Si la función de hash es $h(\text{cédula}) = \text{cédula} \% 10$, se puede almacenar los datos de la siguiente forma:

Índice	Cédula	Nombre	Apellido	Dirección
0				
1	987654321	María	Rodríguez	Heredia
2				
3	456789123	Carlos	Sánchez	Alajuela
4				
5				
6				
7				
8				
9	123456789	Juan	Pérez	San José

La función de hash seleccionada, determina la cantidad de *buckets* (espacios de almacenamiento) que se utilizarán para almacenar los datos. En este caso, se utilizó el módulo 10 para determinar el índice de almacenamiento.

El reto de las funciones hash es generar un índice único para cada llave. Si dos llaves generan el mismo índice, se produce una colisión. Las colisiones se pueden resolver de diferentes formas:

- **Separate chaining:** Cada índice de la tabla *hash* almacena una lista enlazada de elementos que colisionan. Visualmente se puede ver de la siguiente forma:



- **Open addressing:** Se busca un índice alternativo para almacenar el elemento que colisiona.

4.2 Algoritmos de ordenamiento

Son algoritmos que reciben una colección de elementos en desorden y la ordenan ascendente o descendente. Hay muchos algoritmos, y la razón de su existencia es que cada uno tiene diferentes características de rendimiento.

4.2.1 Ordenamiento por selección

Este algoritmo es el más sencillo de implementar. Funciona de la siguiente manera:

1. Busca el elemento más pequeño (o el más grande si es orden descendente) en el arreglo.
2. Intercambia el elemento más pequeño con el primer elemento del arreglo.
3. Busca el segundo elemento más pequeño en el arreglo.

4. Intercambia el segundo elemento más pequeño con el segundo elemento del arreglo.
5. Repite el proceso hasta que el arreglo esté ordenado.

Visualmente se puede ver de la siguiente forma:

values	values	values	values	values
[0] 126	[0] 1	[0] 1	[0] 1	[0] 1
[1] 43	[1] 43	[1] 26	[1] 26	[1] 26
[2] 26	[2] 26	[2] 43	[2] 43	[2] 43
[3] 1	[3] 126	[3] 126	[3] 126	[3] 113
[4] 113	[4] 113	[4] 113	[4] 113	[4] 126
(a)	(b)	(c)	(d)	(e)

Figure 4.1: Ordenamiento por selección

El arreglo se “divide” en dos arreglos “lógicos”:

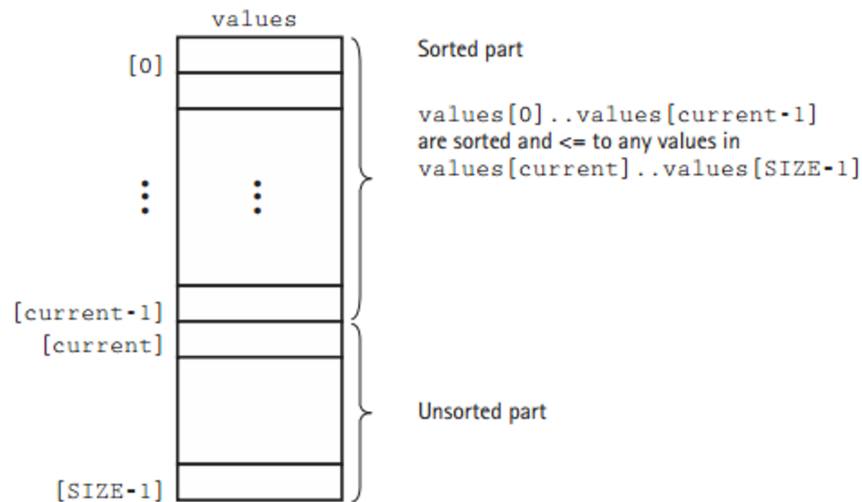


Figure 4.2: División lógica del arreglo

4.2.2 Bubble sort

4.2.3 Ordenamiento por inserción (Insertion sort)

4.2.4 Quick sort

4.2.5 Shellsort

Fue propuesto por Donald Shell en 1959.

La idea básica de Shellsort es que los elementos de un arreglo se ordenan en incrementos cada vez más pequeños. El último incremento es 1, que es el tamaño del arreglo. El algoritmo de Shellsort es una generalización del algoritmo **InsertionSort** y busca aprovechar al máximo el mejor caso de este, es decir, cuando los elementos ya están *casi* ordenados.

Shellsort trabaja realizando sus Insertion Sorts en sublistas cuidadosamente seleccionadas, primero en sublistas pequeñas y luego en sublistas cada vez más grandes.

Shellsort rompe la lista en subconjuntos disjuntos, donde un subconjunto está definido por un “incremento”, I . Cada registro en un subconjunto dado está separado por I posiciones. Por ejemplo, si el incremento fuera 4, entonces cada registro en el subconjunto estaría separado por 4 posiciones.

Usando un enfoque sencillo, podemos definir los gaps (o incrementos) de la siguiente manera:

$\text{size} / 2, \text{size} / 4, \text{size} / 8, \dots, 1$

Rondondeando hacia el entero más cercano hacia arriba. Entonces, dado un array de 9 elementos, los gaps serían:

5, 3, 1

Dado el siguiente arreglo:

Indices	0	1	2	3	4	5	6	7	8	
Elementos	5	3	8	6	2	7	1	4	9	
=====										
gap=5	5	3	8	6	2	7	1	4	9	
		^-----^								
	5	3	8	6	2	7	1	4	9	
		^-----^								
	5	1	8	6	2	7	3	4	9	
		^-----^								
	5	1	4	6	2	7	3	8	9	
		^-----^								
	5	1	4	6	2	7	3	8	9	
=====										
gap de 3		^-----^								
	5	1	4	6	2	7	3	8	9	
		^-----^								
	5	1	4	6	2	7	3	8	9	
		^-----^								
	5	1	4	6	2	7	3	8	9	
		^-----^								
	5	1	4	3	2	7	6	8	9	
		^-----^								
	3	1	4	5	2	7	6	8	9	

[illegible]

4.2.6 Radix sort

Chapter 5

Grafos

Los grafos son estructuras de datos no lineales que consisten de vértices (o nodos) y aristas (o bordes). Un grafo G agrupa entidades físicas o conceptuales. Un grafo está denotado como $G = \{V, A\}$ donde V es el conjunto de vértices y A es el conjunto de aristas.

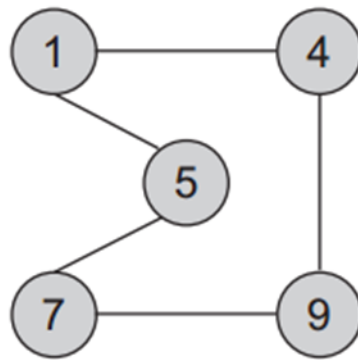


Figure 5.1: Grafo básico

Del grafo anterior, podemos decir que:

- $V = \{ 1, 4, 5, 7, 9 \}$
- $A = \{ (1,4), (4,1), (1,5), (5,1), (4,9), (9,4), (5,7), (7,5), (7,9), (9,7) \}$

Un grafo puede dirigido y no dirigido:

- En los dirigidos se muestra la dirección de la relación entre los nodos.

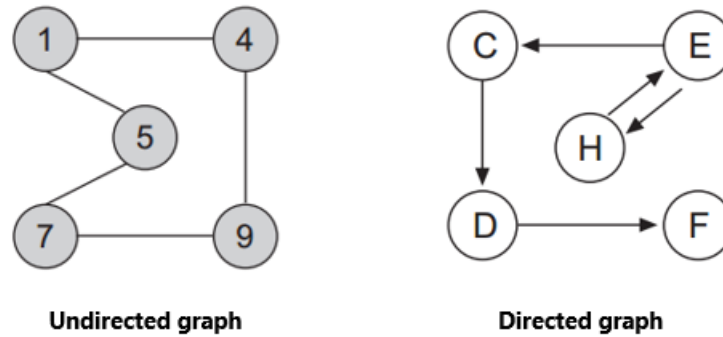


Figure 5.2: Tipos de grafos

- En los no dirigidos los nodos conectados son adyacentes.

5.1 Conceptos importantes

- Una arista puede tener un peso asociado denotando la magnitud asociada a la relación. Este tipo de grafos se les llama *Grafos ponderados*
- El grado de v es una cualidad de un nodo de un grafo. En un grafo no dirigido es el número de aristas que contiene v .
- En un grafo dirigido, el grado de entrada es el número de extremos de cola adyacentes a v . El grado de salida es el número de extremos de cabeza adyacentes a v .
- La ruta $P = (v_0, v_1, v_2, \dots, v_n)$ es una serie de vértices que forman la ruta desde v_0 hasta v_n . v_n y v_0 pueden ser iguales. Si los vértices entre v_0 y v_n son diferentes, la ruta se llama ruta simple.
- Un ciclo es una ruta simple que empieza y termina en el mismo nodo.
- Un DAG es un grafo acíclico dirigido, o sea que no existen ciclos.
- Un grafo es conexo si existe un camino entre cualquier par de nodos que lo componen. Un grafo es fuertemente conexo si el grafo es conexo y es dirigido.

5.2 Representación

Los grafos se pueden representar utilizando dos enfoques diferentes:

- Usando una matriz bidimensional conocida como matriz de adyacencia
- Usando una representación dinámica conocida como lista de adyacencia

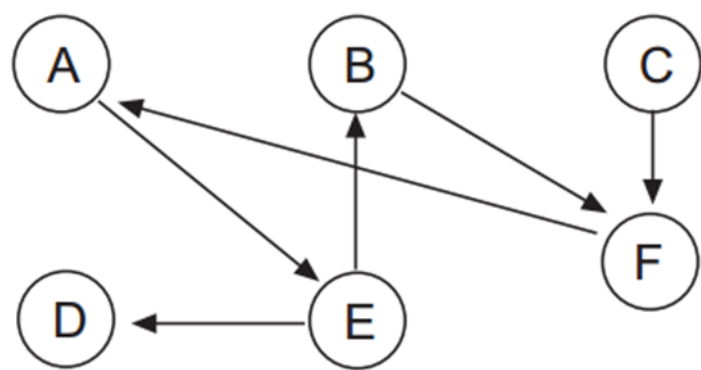


Figure 5.3: ciclo



Figure 5.4: grafo conexo y fuertemente conexo

Elegir entre una representación u otra depende del tipo de array y de las operaciones que se realizarán:

- Si el grafo es denso (muchas aristas), lo mejor es escoger la matriz.
- Si el grafo es disperso, lo mejor es escoger la lista enlazada.

5.2.1 Matriz de adyacencia

Sea $G = \{V, A\}$ donde $V = \{v_0, v_1, v_2, \dots, v_{n-1}\}$ y $A = \{(v_i, v_j)\}$. Los nodos se pueden representar mediante la matriz A de $n \times n$ conocida como matriz de adyacencia. Cada elemento de a_{ij} puede tomar uno de los siguientes valores:

$$a_{ij} \begin{cases} 0 & \text{if there is not an arc between } (v_i, v_j) \\ 1 & \text{if there is an arc between } (v_i, v_j) \end{cases}$$

Figure 5.5: Matriz de adyacencia

- Por ejemplo, digamos que los nodos son $\{D, F, K, L, R\}$ la matriz sería:

$$A = \begin{array}{c|ccccc|} & D & F & K & L & R \\ \hline D & 0 & 1 & 1 & 0 & 0 \\ F & 1 & 0 & 1 & 0 & 0 \\ K & 0 & 0 & 0 & 0 & 0 \\ L & 0 & 1 & 1 & 0 & 0 \\ R & 1 & 0 & 0 & 0 & 0 \end{array}$$

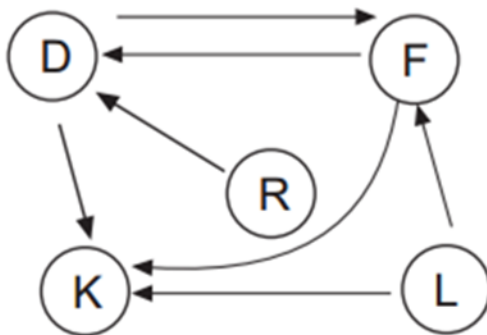


Figure 5.6: alt text

- Si el grafo es ponderado:

$$P = \begin{vmatrix} 0 & 4 & 5 & 0 & 0 \\ 0 & 0 & 3 & 6 & 3 \\ 3 & 0 & 0 & 0 & 0 \\ 5 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{vmatrix}$$

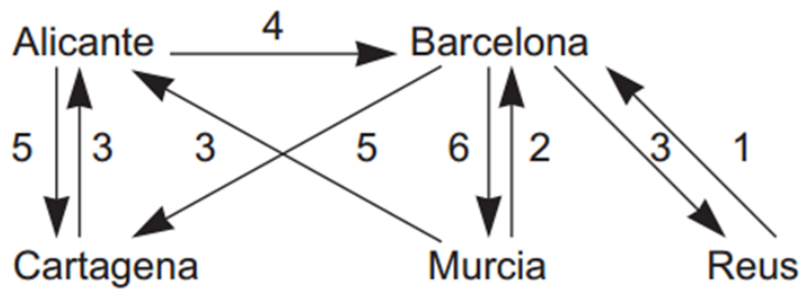


Figure 5.7: matriz grafo ponderado

5.2.2 Lista de adyacencia

Una lista de adyacencia es una lista vinculada donde cada elemento representa un nodo del grafo. Cada elemento contiene una lista de relaciones con otros nodos, siendo el nodo del elemento, el origen.

5.3 Recorridos de un grafo

Atravesar un grafo implica visitar todos los nodos accesibles comenzando desde un nodo específico

Algoritmo de recorrido básico:

- Sea V el conjunto de vértices del gráfico.
- Sea W el conjunto de nodos no visitados. Inicialmente solo contiene el nodo inicial v
- Sea Y el conjunto de nodos visitados.

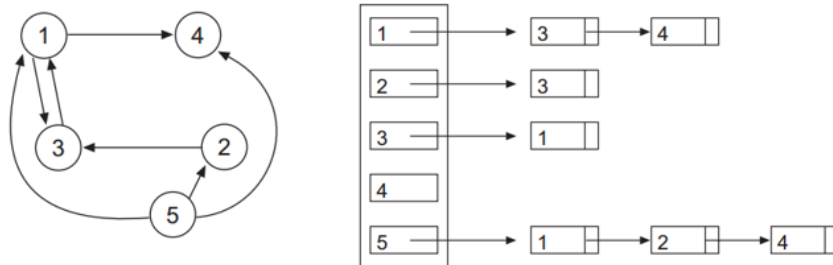


Figure 5.8: Lista de adyacencia

- En cada paso del algoritmo, se elimina un nodo w del conjunto W , se procesa y para cada nodo adyacente de w , si no se ha visitado, se agregará a w . El nodo w se agrega a Y .
- El algoritmo termina cuando W está vacío

5.3.1 Breadth-First

Utiliza una cola que mantiene los vértices marcados.

FIFO logra que a partir de v , primero se procesen todos los vértices adyacentes, luego todos los adyacentes de los adyacentes de v ... y así sucesivamente

Algoritmo:

1. Marque el nodo de inicio v
2. Poner en cola el nodo de inicio v
3. Repita los pasos 4 y 5 hasta que la cola esté vacía.
4. Sacar de cola el nodo w de la cola, procesar w
5. Ponga en cola todos los nodos adyacentes a w que no estén marcados, nodos en cola marcados
6. Fin

5.3.2 Depth-First

En Depth-First, el orden de procesamiento viene dado por un enfoque LIFO

Atravesar el grafo con un nodo v . v se marca como visitado y se empuja a la pila. La parte superior de la pila está reventada.

Cada nodo adyacente de v no visitado se empuja a la pila.

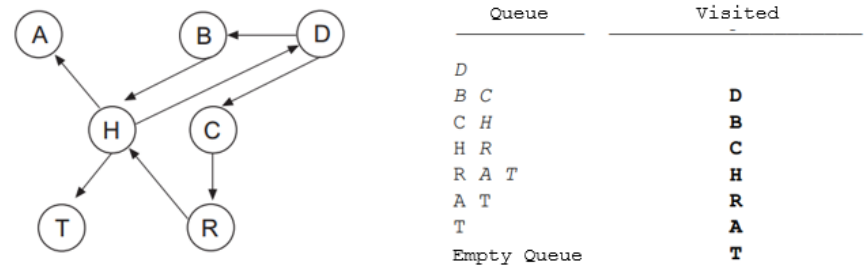


Figure 5.9: breadth first alg

Esto continua hasta que no haya más elementos en la pila.

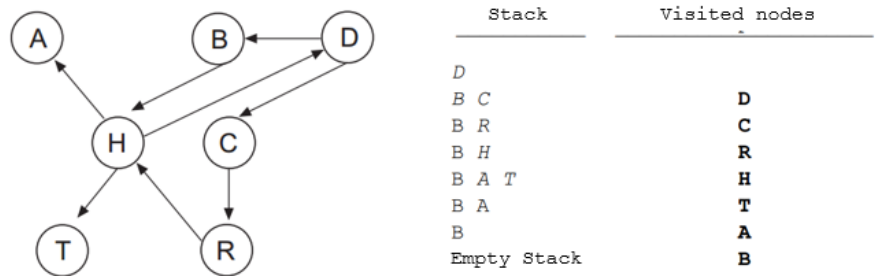


Figure 5.10: depth first

5.4 Camino más corto: Dijkstra

Uno de los problemas más comunes es determinar el camino más corto entre un par de nodos. Para este tipo de problema consideramos un grafo dirigido y ponderado.

La longitud del camino más corto es la suma del peso de cada arista. El algoritmo de Dijkstra encuentra el camino más corto desde un nodo de origen a todos los demás nodos en un gráfico con pesos positivos

Edsger Dijkstra (1930 - 2002) fue un informático holandés que dio forma a la programación informática como una ciencia reconocida.

¿Cómo funciona?

- 1. Se utilizará una tabla donde la primera columna es el vertice, la segunda es el peso temporal que se le dará a un camino y en la tercera columna el peso final.

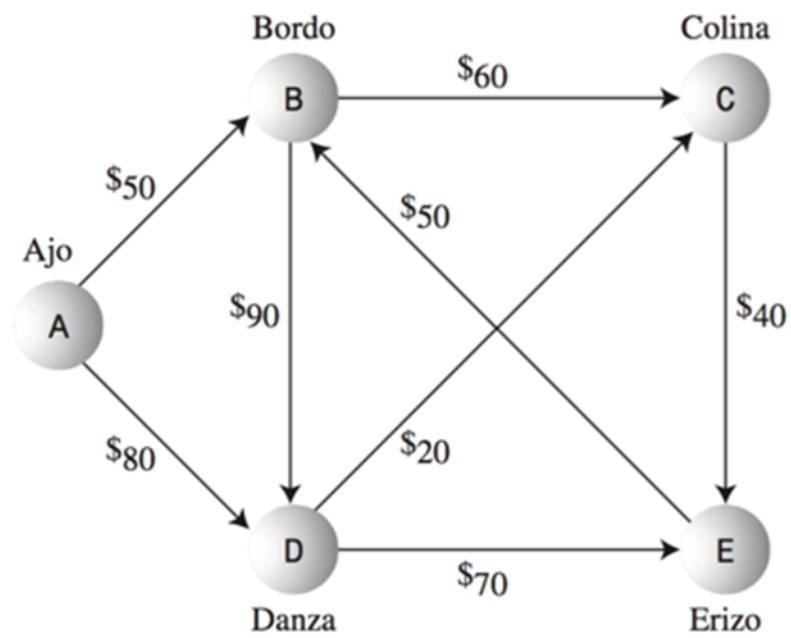


Figure 5.11: Dijkstra technique

Vertice	Temporal	Final
A	0	0
B	50	50
C	110	110
D	80	80
E	150	150

2. Escoger un punto de salida y otro de entrada. A -> E.
3. Empezar por los nodos adyacentes al de salida y analizando su peso, este se coloca en la columna temporal.
4. Se debe escoger el de magnitud más corta y se coloca este número en peso final. A partir de este nodo se le analizan sus adyacentes pero la magnitud es la suma del peso final + el peso que exista en las aristas.
5. Se analiza cual es el menor y a partir de ahí se sigue analizando. *En este caso la distancia mas corta es A -> D -> E*

5.4.1 Estructura general

Foreach node set distance [node] = HIGH

SettledNodes = empty

UnSettledNodes = empty

Add sourceNode to UnSettledNodes

distance[sourceNode] = 0

while (UnSettledNodes is not empty) {

 evaluationNode = getNodeWithLowestDistance(UnSettledNodes)

 remove evaluationNode from UnSettledNodes

 add evaluationNode to SettledNodes evaluatedNeighbors(evaluationNode)

}

getNodeWithLowestDistance(UnSettledNodes){

 find the node with the lowest distance in UnSettledNodes and return it

}

evaluatedNeighbors(evaluationNode){

 Foreach destinationNode which can be reached via an edge from evaluationNode AND which is not

 edgeDistance = getDistance(edge(evaluationNode, destinationNode))

 newDistance = distance[evaluationNode] + edgeDistance

 if (distance[destinationNode] > newDistance) {

 distance[destinationNode] = newDistance

 evaluation.predecessor = evaluationNode

 add destinationNode to UnSettledNodes

 }

}

}

5.5 Camino más corto: Floyd

¿Cómo calcular el camino más corto de cada nodo a cada nodo? Existe una solución más directa que usar dijkstra de nodo en nodo.

El algoritmo de Floyd calcula mediante programación dinámica el camino más corto de cada nodo a cada nodo.

El algoritmo de Floyd representa el gráfico como una matriz ponderada. Cada arco (v_i, v_j) tiene un peso c_{ij} . Si el arco no existe, el valor es infinito. La diagonal de la matriz es igual a cero.

El algoritmo de Floyd determina una nueva matriz D de $n \times n$ elementos, donde cada D_{ij} es el camino mínimo de v_i a v_j

- En cada paso desde D_0 se genera una nueva matriz $D_1, D_2, \dots, D_k, D_n$. En cada paso se incluye un nuevo vértice para determinar si ese vértice mejora los caminos para que sean más cortos.

$$D_0[i, j] = \begin{cases} c_{ij} & \text{weight of the arc from vertex } i \text{ to vertex } j \\ \infty & \text{if there is no arc} \end{cases}$$

$$D_1[i, j] = \text{minimum}(D_0[i, j], D_0[i, 1] + D_0[1, j])$$

$$D_2[i, j] = \text{minimum}(D_1[i, j], D_1[i, 2] + D_1[2, j])$$

$$D_k[i, j] = \text{minimum}(D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j])$$

Figure 5.12: D matrix

- Otra matriz $Q_1, Q_2, \dots, Q_k, Q_n$ se genera en cada paso desde Q_0 . Q es la matriz predecesora.
-

$$Q_0[i, j] = \begin{cases} 0 & \text{if } i = j \text{ or } D_{ij} = \infty \\ i & \text{in all other cases} \end{cases}$$

$$Q_n[i, j] = \begin{cases} Q_{n-1}[i, j] & \text{if } D_{n-1}[i, j] \leq D_{n-1}[i, n] + D_{n-1}[n, j] \\ Q_{n-1}[n, j] & \text{if } D_{n-1}[i, j] > D_{n-1}[i, n] + D_{n-1}[n, j] \end{cases}$$

Figure 5.13: Q matrix

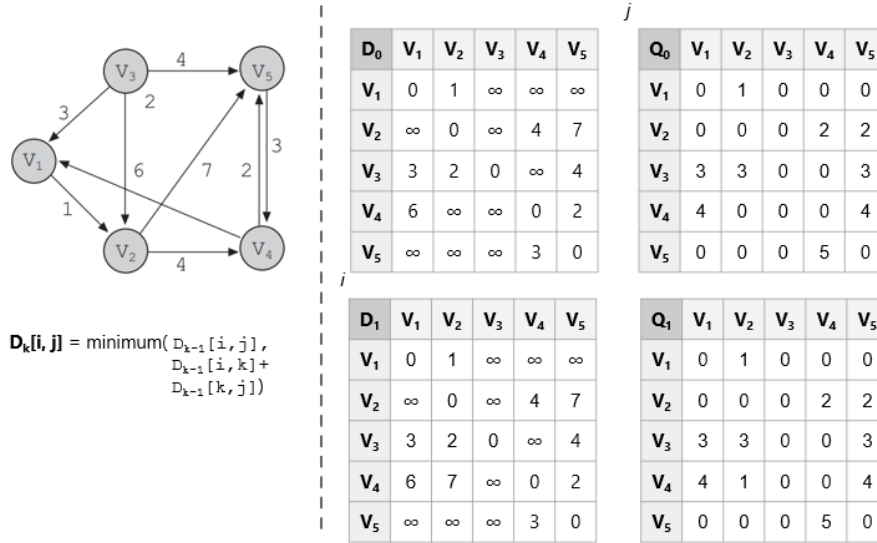


Figure 5.14: Floyd alg

5.6 Warshall

Similar al algoritmo de Floyd. Calcula la matriz de camino P (también llamada cierre transitivo) de un grafo G de n vértices, representado por su matriz de adyacencia A.

Define una secuencia de matrices nxn $P_0, P_1, P_2, P_3, \dots, P_n$

$$\begin{aligned}
 P_0[i, j] &= \begin{cases} 1 & \text{if there is an arc from } v_i \text{ to } v_j \\ 0 & \text{if there is no arc} \end{cases} \\
 P_1[i, j] &= \begin{cases} 1 & \text{if there is an arc from } v_i \text{ to } v_j \text{ that does not go through} \\ & \text{other vertex different of } v_1 \\ 0 & \text{if there is no arc} \end{cases} \\
 P_2[i, j] &= \begin{cases} 1 & \text{if there is an arc from } v_i \text{ to } v_j \text{ that does not go through} \\ & \text{other vertex different of } v_1 \dots v_2 \\ 0 & \text{if there is no arc} \end{cases} \\
 P_n[i, j] &= \begin{cases} 1 & \text{if there is an arc from } v_i \text{ to } v_j \text{ that does not go through} \\ & \text{other vertex different of } v_1 \dots v_n \\ 0 & \text{if there is no arc} \end{cases}
 \end{aligned}$$

Figure 5.15: Warshall algorithm

5.7 Minimal Spanning tree

Se utiliza un grafo no dirigido para modelar relaciones simétricas entre vértices del gráfico. Cualquier arco (v, w) de un grafo no dirigido es igual que el arco de (w, v) .

Una tarea común es determinar si para cualquier par de vértices existe un camino que los conecte, es decir, **si es un grafo conexo**.

Un *árbol de expansión mínimo* es un subconjunto del grafo que cubre todos los vértices y cuyos bordes tienen una suma de los pesos mínimos. - Aplicado en redes

Un **árbol** es un subconjunto del grafo que está conexo y no tiene ciclos. - Si tiene n vértices, entonces tiene n-1 aristas. - Existe un camino único entre dos vértices cualesquiera de un árbol. - Si se agrega una ventaja, se produce un ciclo.

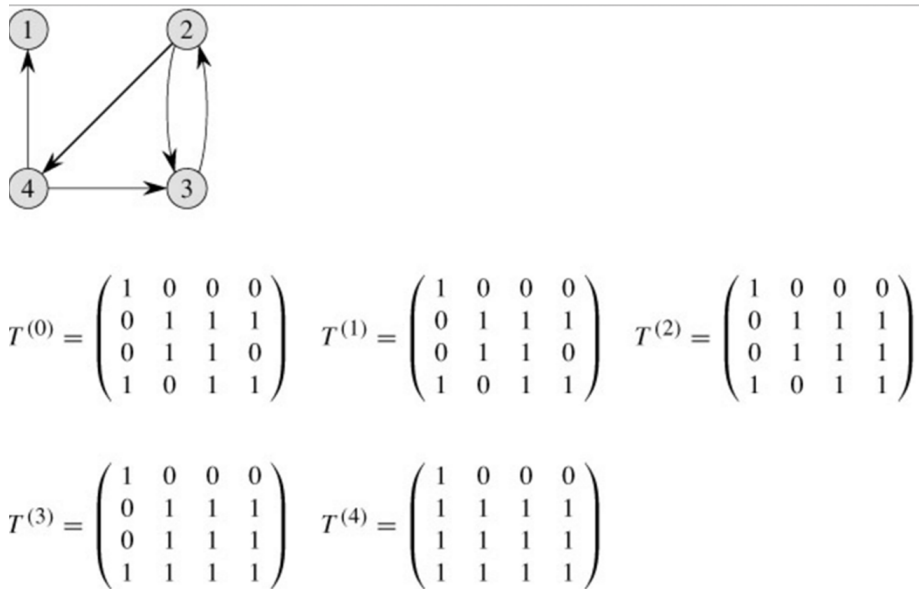


Figure 5.16: warshall algorithm2

Si todos los vértices están en el árbol, entonces es un grafo conexo.

Dado un grafo no dirigido, encuentre el árbol de expansión mínimo

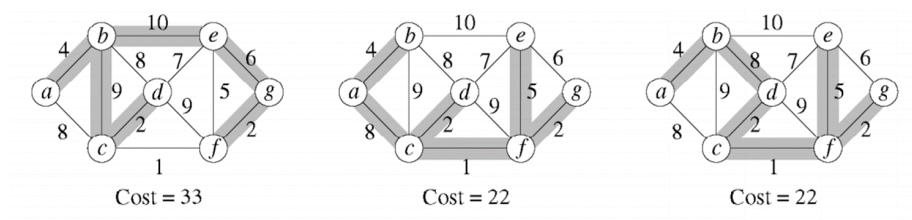


Figure 5.17: minimal spanning tree

- El mismo grafo puede tener varios arboles de expansión, pero no todos son el mínimo.

¿Como conseguirlo?

- Prim algorithm
- Kruskal algorithm

5.7.1 Prim

El árbol crece en etapas sucesivas. En cada etapa, se elige un nodo como raíz y agregamos un borde y el vértice asociado al árbol.

En cualquier punto tenemos un conjunto de vértices que ya han sido incluidos en el árbol.

El algoritmo encuentra un nuevo vértice para agregar al árbol eligiendo el borde (u,v) , tal como el costo de (u,v) es el más pequeño entre todos los bordes donde u está en el árbol y v no.

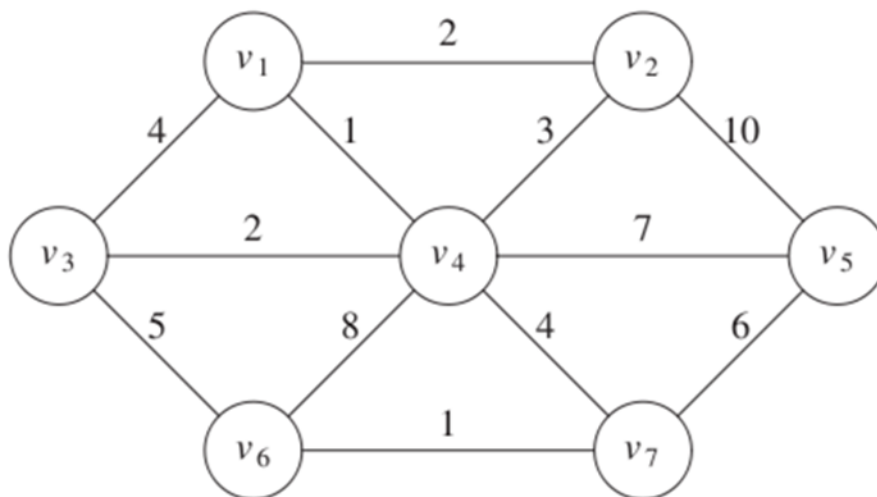


Figure 5.18: prim 1

Peso mínimo conectado a un nodo conocido

Al realizar todo el algoritmo el resultado se vería así:

5.7.2 Kruskal

Selecciona bordes si el orden es de menor peso y acepta un borde si no causa un ciclo

Mantiene un bosque (colección de árboles). Inicialmente todos son árboles de un solo nodo. Agregar un borde fusiona dos árboles en uno

Si u y v están en el mismo conjunto, la arista (u,v) se rechaza, porque sumarla causaría un ciclo. u y v están en el mismo conjunto si están conectados.

v	$known$	d_v	p_v
v_1	F	0	0
v_2	F	∞	0
v_3	F	∞	0
v_4	F	∞	0
v_5	F	∞	0
v_6	F	∞	0
v_7	F	∞	0

Figure 5.19: prim table

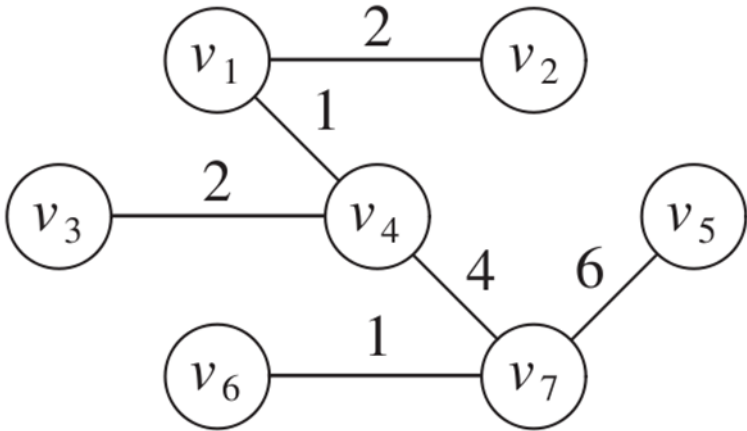


Figure 5.20: prim result

v	$known$	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	2	v_4
v_4	T	1	v_1
v_5	T	6	v_7
v_6	T	1	v_7
v_7	T	4	v_4

Figure 5.21: prim table result

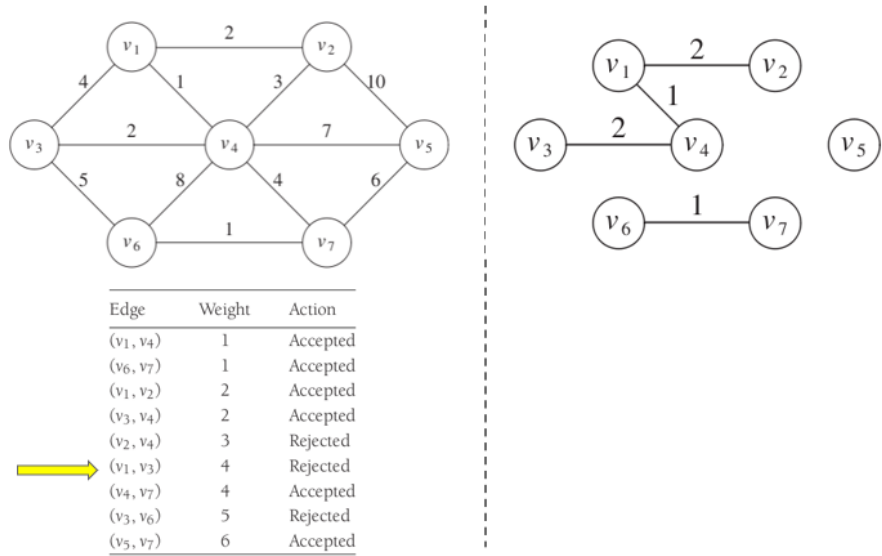


Figure 5.22: Kruskal